



UNIVERSIDA TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO
DÍGITAL
INGENIERÍA DE SOFTWARE



ASIGNATURA:

APLICACIONES WEB

TEMA:

PRACTICA EXPERIMENTAL UNIDAD II

DOCENTE:

GUERRERO ULLOA GLEISTON CICERON

INTEGRANTES:

DARWIN ORLANDO ARCALLE GREFA
ZAMORA ARIAS CARLA ESTHEFANIA

NIVEL:

5TO SOFTWARE

PARALELO:

“B”

FECHA:

11/06/2026

GitHub:

https://github.com/carla22072004/Practica-EXPERIMENTAL_II.git

ÍNDICE DE CONTENIDOS

ÍNDICE DE CONTENIDOS	2
1. INTRODUCCIÓN	3
2. TEMA 1: INTRODUCCIÓN A LA PROGRAMACIÓN DEL LADO DEL SERVIDOR	3
2.1 Programación del Lado del Servidor: Conceptos Generales	3
2.2 PERL para Aplicaciones Web	5
2.3 PHP: Hypertext Preprocessor	7
2.4 Comparativa Tecnológica y Criterios de Selección	11
3. TEMA 2: DESARROLLO WEB CON JAVA Y JSP	13
3.1 Java para el Desarrollo Web: Plataforma y Herramientas	13
3.2 Java Servlets	15
3.3 JavaServer Pages (JSP)	18
3.4 Introducción a Spring Boot para Desarrollo Web	20
4. TEMA 3: DESARROLLO WEB CON ASP.NET	23
4.1 La Plataforma .NET y ASP.NET	23
4.2 ASP.NET Core: Estructura de un Proyecto Web	24
4.3 Razor Pages y Razor Views	27
4.4 Formularios, Validación y Seguridad en ASP.NET	29
5. TEMA 4: SEGURIDAD Y BUENAS PRÁCTICAS EN PROGRAMACIÓN WEB	32
5.1 Seguridad en Aplicaciones Web	32
5.2 OWASP Top 10 y Contramedidas	34
5.3 Buenas Prácticas de Codificación Web	36
6. APLICACIÓN DINÁMICA: CRUD CON AUTENTICACIÓN	38
6.1 Descripción de la Aplicación	38
6.2 Implementación en PHP	39
6.3 Implementación en ASP.NET Core	42
6.4 Controles OWASP Implementados	44
6.5 Análisis Comparativo de Entornos Tecnológicos	46
7. CONCLUSIONES	49
9. REFERENCIAS	50
10. ANEXOS	52
Anexo A: Commits – GitHub	52
Anexo B: Capturas de Pantalla de la Aplicación	53

1. INTRODUCCIÓN

En el ecosistema tecnológico contemporáneo, el desarrollo de aplicaciones web ha migrado desde la simple exhibición de documentos estáticos hacia la construcción de complejas plataformas interactivas, capaces de procesar volúmenes masivos de datos en tiempo real. Detrás de esta interfaz visual con la que interactúa el usuario final, se erige la programación del lado del servidor (backend), una disciplina infraestructural encargada de gobernar la lógica de negocio, administrar la persistencia en sistemas de bases de datos, gestionar el estado de las sesiones y garantizar la seguridad de la información. La correcta selección y el dominio del stack tecnológico del lado del servidor representan decisiones estratégicas críticas que determinan la escalabilidad, el rendimiento y la viabilidad a largo plazo de cualquier proyecto de software.

Esta práctica experimental se ofrece un análisis exhaustivo y estructurado de los principales ecosistemas que sustentan el desarrollo web backend actual que abarca toda la unidad 2. El recorrido inicia con una revisión histórica de tecnologías fundamentales como Perl y la evolución del omnipresente PHP, evaluando sus ciclos de vida y capacidades de abstracción segura a través de PDO. Posteriormente, el estudio se adentra en entornos de grado corporativo, analizando el robusto estándar de Jakarta EE (Servlets y JSP) y su transición hacia la agilidad moderna que provee Spring Boot. De igual manera, se examina la madurez de la plataforma .NET de Microsoft, contrastando el flujo modular de procesamiento de datos por capas (middleware) de ASP.NET Core frente a sus modelos de presentación visual como Razor.

Finalmente, reconociendo que la funcionalidad carece de valor si se encuentra expuesta a amenazas informáticas, el documento dedica un bloque riguroso a la seguridad de la información. Tomando como base los marcos de trabajo y directrices de instituciones de prestigio internacional como el NIST y la Fundación OWASP, se detallan las contramedidas de código necesarias para neutralizar los vectores de ataque más críticos de la industria. Para consolidar este marco teórico, la sección de cierre detalla el diseño arquitectónico e implementación práctica de una aplicación dinámica funcional (CRUD con autenticación), demostrando la convergencia real entre la teoría de la ingeniería de software y las demandas de seguridad exigidas en los entornos de producción actuales.

2. TEMA 1: INTRODUCCIÓN A LA PROGRAMACIÓN DEL LADO DEL SERVIDOR

2.1 Programación del Lado del Servidor: Conceptos Generales

La programación del lado del servidor (Server-Side Programming) consiste en el desarrollo de aplicaciones cuya lógica principal se ejecuta en un servidor web antes de enviar una respuesta al navegador del usuario. A diferencia de la programación del lado del cliente, donde el código es procesado por el navegador mediante tecnologías como JavaScript, el código del lado del servidor permanece oculto al usuario y es responsable del procesamiento de datos, autenticación, acceso a bases de datos, gestión de sesiones, generación dinámica de contenido y comunicación con otros servicios.

Cuando un usuario solicita una página web, el servidor recibe la petición mediante el protocolo HTTP o HTTPS. Posteriormente, el lenguaje de programación del servidor ejecuta la lógica correspondiente, consulta la base de datos si es necesario, procesa la información y genera una respuesta que normalmente consiste en HTML, CSS, JavaScript o datos en formato JSON para aplicaciones modernas [1].

El flujo de trabajo fundamental sigue estos pasos:

- **Petición (Request):** El cliente (navegador, app móvil) envía una solicitud al servidor con una URL y un método específico (GET, POST, PUT, DELETE).

- **Procesamiento:** El servidor web intercepta la petición, el lenguaje de programación ejecuta la lógica correspondiente y, si es necesario, interactúa con el sistema de gestión de bases de datos (RDBMS o NoSQL).
- **Respuesta (Response):** El servidor empaqueta el resultado junto con un código de estado HTTP (ej. 200 OK, 404 Not Found) y lo envía de vuelta al cliente.

Las principales funciones de la programación del lado del servidor incluyen:

Función	Descripción	Ejemplo Práctico
Procesamiento de formularios	Recepción, lectura y tratamiento de los datos enviados por el usuario desde elementos de la interfaz [2].	Capturar los campos de un formulario de registro (nombre, correo) cuando el usuario hace clic en "Enviar".
Validación de datos	Verificación estricta en el servidor de que los datos recibidos cumplen con los formatos, tipos y restricciones requeridos. <i>(Nota: La validación en el cliente es estética; la del servidor es obligatoria por seguridad)</i> [2].	Comprobar que un correo electrónico tenga un formato válido y que la contraseña cumpla con la longitud mínima antes de guardarla.
Gestión de usuarios y autenticación	Control de acceso al sistema, verificación de identidades y asignación de permisos (roles) [2].	Comparar la contraseña ingresada con el hash encriptado guardado en la base de datos para permitir el inicio de sesión.
Administración de sesiones	Mecanismo para "recordar" al usuario a través de múltiples peticiones HTTP (las cuales, por naturaleza, no tienen estado) [1].	Mantener el carrito de compras de un usuario activo mientras navega por diferentes páginas de una tienda en línea.
Acceso a bases de datos	Conexión, consulta, inserción, actualización y eliminación de información en sistemas de almacenamiento persistente [2].	Realizar una consulta SQL para traer los últimos 10 artículos publicados en un blog.
Implementación de reglas de negocio	Aplicación de la lógica, cálculos y restricciones específicas que definen cómo opera la empresa o el sistema.	Calcular el descuento de un producto basado en los puntos de fidelidad del cliente y los impuestos locales.
Desarrollo de APIs REST y servicios web	Creación de puntos de acceso (endpoints) para que diferentes aplicaciones o el propio frontend (Angular, React, etc.) consuman datos de manera desacoplada [3].	Retornar una lista de productos en formato JSON para que sea renderizada por una aplicación móvil nativa.
Seguridad de la aplicación	Protección contra vulnerabilidades comunes (como inyecciones SQL, XSS, CSRF) y cifrado de datos sensibles [4].	Sanitizar las entradas de texto y asegurar que las conexiones viajen cifradas mediante certificados SSL/TLS.

Tabla 1. Funciones principales del lado del servidor.

2.2 PERL para Aplicaciones Web

PERL (Practical Extraction and Reporting Language) es un lenguaje interpretado de propósito general creado por Larry Wall en 1987. Diseñado en sus inicios como una herramienta optimizada para la administración de sistemas Unix y el procesamiento avanzado de texto [5], su flexibilidad provocó que los primeros desarrolladores web lo adoptaran de forma masiva en la década de los 90. Esto lo convirtió en la primera tecnología dominante para la generación de contenido dinámico en la incipiente World Wide Web.

Si bien la madurez del lenguaje está respaldada por la comunidad a través de repositorios globales de módulos como CPAN (Comprehensive Perl Archive Network) [6], su evolución frente a tecnologías modernas presenta marcadas ventajas y limitaciones en el contexto actual del desarrollo web [7]:

- **Ventajas:** Sobresale por su potente motor nativo de expresiones regulares para la manipulación y análisis de cadenas de texto [5]. Es un entorno altamente estable, multiplataforma y con una inmensa retrocompatibilidad que garantiza que el código escrito hace décadas siga funcionando con un mantenimiento mínimo.
- **Limitaciones:** Su filosofía de diseño ("There's more than one way to do it" o hay más de una forma de hacerlo) fomenta una sintaxis extremadamente flexible pero compacta que suele derivar en código difícil de leer o mantener por terceros ("código críptico") [5]. Adicionalmente, posee una curva de aprendizaje elevada para programadores noveles y carece del ecosistema masivo de herramientas de desarrollo ágil que hoy disfrutan entornos como Node.js, Python o PHP [3].

Por otro lado, la estructura de datos de Perl se fundamenta en tres tipos de variables nativas, diferenciadas por caracteres especiales (sigilos), las cuales se aplican directamente para estructurar respuestas y manejar configuraciones web [5], [6]:

- **Escalares (\$):** Almacenan datos únicos como cadenas de texto, números o referencias (ej. \$nombre_usuario = "Juan";).
- **Arrays (@):** Listas ordenadas de datos indexados (ej. @metodos_http = ("GET", "POST");).
- **Hashes (%):** Arreglos asociativos de clave y valor, ideales para mapear cabeceras HTTP o respuestas estructuradas (ej. %cabecera = ("Content-Type" => "text/html");).

A continuación, se presenta un script básico en Perl estructurado para ejecutarse en un entorno web tradicional:

```
#!/usr/bin/perl
# Línea Shebang: indica al servidor la ruta del intérprete de Perl

use strict;      # Fuerza la declaración explícita de variables para evitar
errores
use warnings;    # Activa alertas detalladas en el log del servidor durante la
ejecución

# Declaración de variables básicas
my $titulo_sitio = "Mi Primera Aplicación en Perl";
my @tecnologias  = ('Perl', 'CGI', 'Apache');

# Impresión de la cabecera HTTP obligatoria
print "Content-Type: text/html; charset=UTF-8\n\n";

# Generación del cuerpo del documento HTML
```

```
print "<!DOCTYPE
html>\n<html>\n<head><title>$titulo_sitio</title></head>\n<body>";
print "<h1>Bienvenido al Servidor</h1>";
print "<p>Entorno web generado dinámicamente con las siguientes
herramientas:</p>";
print "<ul>";
foreach my $tech (@tecnologias) {
    print "<li>$tech</li>";
}
print "</ul>\n</body>\n</html>";
```

En cuanto al protocolo CGI (Common Gateway Interface) es un estándar que define como un servidor web (como Apache o Nginx) interactúa con programas externos para generar contenido dinámico [1]. Perl se convirtió en el lenguaje estándar para escribir estos scripts debido a su capacidad para procesar los flujos de datos del servidor.

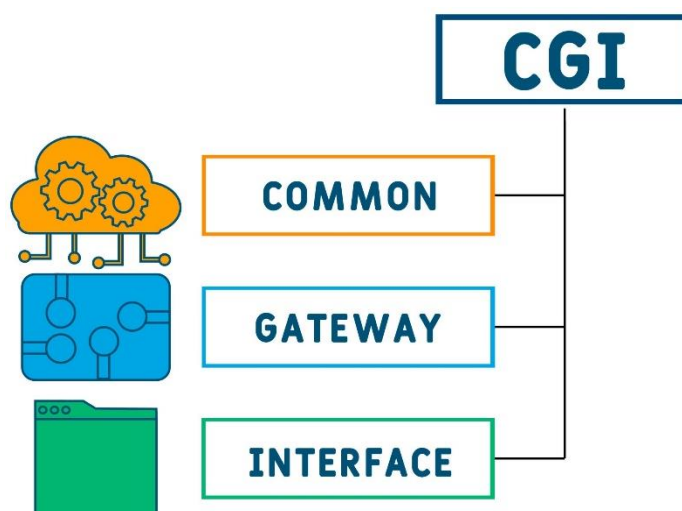


Fig. 1. Arquitectura del Modelo de Ejecución CGI tradicional.
Fuente: Nadezhda Kozhedub / Getty Images.

Bajo el modelo tradicional de CGI, el proceso sigue un flujo síncrono muy estricto [1]:

- El navegador del cliente realiza una petición HTTP solicitando un archivo ejecutable (ej. index.cgi).
- El servidor web recibe la solicitud y, en lugar de leer un archivo estático, crea un nuevo proceso en el sistema operativo para lanzar el intérprete de Perl.
- El script de Perl lee las variables de entorno suministradas por el servidor (como los datos del formulario), procesa la lógica y escribe el resultado directamente en la salida estándar (STDOUT).
- El servidor web toma ese flujo de texto, le añade las cabeceras de red correspondientes y lo envía de regreso al cliente.

El problema del rendimiento en CGI: Cada petición HTTP entrante requiere que el sistema operativo inicialice un proceso completamente nuevo desde cero para ejecutar el intérprete de Perl. En entornos con alto tráfico, este modelo satura rápidamente la memoria RAM y la CPU del servidor, lo que motivó la migración hacia arquitecturas de procesos persistentes como FastCGI, mod_perl o los servidores de aplicaciones integrados de la actualidad [7], [1].

En la actualidad, Perl ha cedido el protagonismo en el desarrollo de nuevas plataformas web comerciales frente a frameworks de Javascript o Python [3]. No obstante, mantiene una relevancia crítica en áreas especializadas de la industria tecnológica actual. Como en el mantenimiento de Sistemas Heredados (Legacy Systems) en el que Grandes corporaciones financieras, de telecomunicaciones e instituciones gubernamentales operan sobre plataformas backend robustas construidas en Perl que no se reemplazan debido al altísimo costo y riesgo operativo de una migración [7], [6].

También en la automatización y Devops, su integración nativa en entornos Unix/Linux lo convierte en una herramienta idónea para la creación de scripts de administración de servidores, tareas cronometradas de limpieza de sistemas y canalizaciones de despliegue automatizado (pipelines) y frameworks modernos en el que a pesar del declive de CGI, la comunidad de Perl desarrolló frameworks modernos basados en arquitecturas orientadas a eventos y patrones MVC (Modelo-Vista-Controlador) como Mojolicious, Dancer2 y Catalyst, los cuales implementan servidores embebidos y conexiones WebSockets capaces de competir en rendimiento con soluciones contemporáneas [7].

2.3 PHP: Hypertext Preprocessor

PHP (PHP: Hypertext Preprocessor, originalmente Personal Home Page) es un lenguaje de programación interpretado de código abierto, diseñado específicamente para el desarrollo web del lado del servidor. Creado por Rasmus Lerdorf en 1995, nació como una serie de scripts en C para rastrear las visitas de su currículum en línea. A diferencia de lenguajes de propósito general adaptados a la web, PHP se concibió desde su origen para interactuar de forma nativa con el protocolo HTTP y embeberse directamente dentro del código HTML [2].

A lo largo de las décadas, el lenguaje ha sufrido una metamorfosis profunda. Con el lanzamiento de PHP 8.x, el ecosistema se alejó definitivamente de la reputación de lenguaje débilmente tipado y lento. Las versiones actuales incorporan un motor de compilación Just-In-Time (JIT) que mejora el rendimiento computacional en operaciones intensivas [7], un sistema de tipado fuerte y estricto, propiedades promovidas en constructores, enumeraciones nativas y atributos (metadatos) [2].

Su arquitectura destaca por una integración limpia con servidores web a través de módulos como `mod_php` en Apache o mediante el administrador de procesos FPM (FastCGI Process Manager) en servidores modernos como Nginx [1]. Además, posee una capa de abstracción nativa para interactuar con prácticamente cualquier sistema de gestión de bases de datos relacionales (RDBMS) o NoSQL, consolidándose como el motor principal detrás de los sistemas de gestión de contenidos (CMS) más grandes del mundo (WordPress, Drupal) y frameworks de grado empresarial como Laravel y Symfony [3].

A diferencia de entornos que mantienen un servidor de aplicaciones persistente en memoria (como Node.js o Java), PHP funciona bajo un modelo de ejecución de ciclo corto orientado a la petición (Request-Driven Architecture) [1]. El ciclo de vida de cada script sigue una secuencia de fases estrictas por cada petición HTTP entrante [2], [1]:

- **Petición y Arranque (Startup):** El servidor web (ej. Nginx) transfiere la solicitud HTTP a PHP-FPM. PHP inicializa sus módulos principales y extensiones configuradas en el archivo `php.ini`.
- **Compilación a OpCodes:** El motor Zend Engine lee el script `.php` solicitado. El analizador léxico y sintáctico traduce el código fuente legible en instrucciones binarias intermedias conocidas como OpCodes (Códigos de Operación). (Nota: Si OPcache está activo, este paso se omite y los OpCodes se leen directamente desde la memoria RAM, acelerando el proceso).

- **Ejecución:** La máquina virtual de Zend ejecuta los OpCodes. En esta fase se realizan las consultas a bases de datos, se procesan las reglas de negocio y se genera el buffer de salida (generalmente texto HTML o JSON).
- **Finalización (Shutdown):** Se envía la respuesta generada de vuelta al servidor web (y este al cliente). Acto seguido, PHP destruye todas las variables, libera la memoria asignada, cierra las conexiones a bases de datos no persistentes y el entorno queda completamente limpio para la siguiente petición.

La sintaxis de PHP hereda una estructura fuertemente influenciada por C, C++ y Java.

Componente	Categoría / Tipo	Descripción Técnica	Ejemplo en Código
Variables y Tipado	Declaración General	Se identifican con el símbolo \$. Son de tipado dinámico por defecto (el tipo se define según el contexto del valor asignado) [5].	<pre>\$usuario = "Ana"; \$edad = 28;</pre>
	Tipado Estricto	Introducido en versiones modernas. Fuerza al motor de PHP a validar de forma estricta los tipos en argumentos y retornos de funciones [5].	<pre>declare(strict_types=1);</pre>
Tipos de Datos	Escalares (Primitivos)	Contienen un único valor básico: <code>bool</code> (booleano), <code>int</code> (entero), <code>float</code> (punto flotante) y <code>string</code> (cadena de texto).	<pre>\$activo = true; \$precio = 19.99;</pre>
	Compuestos	Almacenan colecciones de datos o estructuras complejas: <code>array</code> (arreglos indexados o asociativos) y <code>object</code> (instancias de clases).	<pre>\$colores = ['azul', 'rojo']; \$user = new Usuario();</pre>
	Especiales	Tipos para escenarios de control específicos: <code>null</code> (ausencia de valor) y <code>resource</code>	<pre>\$resultado = null; \$archivo = fopen("log.txt", "r");</pre>

Componente	Categoría / Tipo	Descripción Técnica	Ejemplo en Código
		(referencias a recursos externos del sistema) [5].	
Estructuras de Control	Condicionales	Bifurcan el flujo de ejecución evaluando expresiones lógicas. Incluye el clásico <code>if / else</code> y evaluaciones múltiples con <code>switch</code> (o <code>match</code> en PHP 8+) [5].	<pre>if (\$edad >= 18) { ... } else { ... }</pre>
	Bucles Tradicionales	Ciclos repetitivos basados en contadores o condiciones que deben cumplirse (<code>for</code> , <code>while</code> , <code>do-while</code>).	<pre>for (\$i = 0; \$i < 10; \$i++) { ... }</pre>
	Bucle de Colecciones	El bucle <code>foreach</code> . Es la estructura más eficiente, nativa y optimizada para recorrer arreglos asociativos e iterar sobre objetos [2], [5].	<pre>foreach (\$productos as \$clave => \$valor) { ... }</pre>

Tabla 2. Variables, tipos de datos y estructuras de control.

Un ejemplo de sintaxis moderna y limpia aplicada en backend:

```
<?php
declare(strict_types=1);

// Definición de una función con tipado estricto (PHP 8+)
function calcularImpuesto(float $monto, float $tasa = 0.12): float {
    return $monto * $tasa;
}

$precioProducto = 150.50;
$totalImpuesto = calcularImpuesto($precioProducto);

// Estructura de control condicional clásica
if ($totalImpuesto > 15.0) {
    echo "El impuesto aplicado es alto.";
} else {
    echo "El impuesto aplicado es estándar.";
}
```

`?>`

PHP simplifica la captura de datos HTTP transformando los parámetros de las peticiones directamente en variables superglobales (arreglos asociativos disponibles en cualquier parte del código) [2]:

- **\$_POST:** Almacena los datos enviados a través del cuerpo de una petición HTTP POST (ideal para formularios de inserción o credenciales).
- **\$_GET:** Almacena los parámetros pasados de forma visible a través de la URL de la petición.

```
// Captura segura de un formulario básico
$nombreUsuario = $_POST['txt_usuario'] ?? 'Invitado'; // Operador de fusión nula
```

Debido a que el protocolo HTTP es stateless (sin estado), PHP incorpora un sistema nativo para la Administración de Sesiones [1], [2]. Al invocar `session_start()`, el servidor genera un identificador único aleatorio (Session ID). Este ID se envía automáticamente al navegador del usuario en una cookie llamada `PHPSESSID`.

En las siguientes peticiones, el navegador devuelve dicha cookie, permitiendo a PHP reconstruir el arreglo superglobal `$_SESSION` para recuperar los datos persistentes del usuario (como su estado de autenticación o carrito de compras) almacenados de forma segura en el disco duro del servidor.

Para interactuar con sistemas de bases de datos como MySQL, PHP ofrece dos extensiones principales de manera nativa [2]:

- **MySQLi (MySQL Improved):** Diseñada exclusivamente para bases de datos MySQL. Permite un enfoque tanto procedural como orientado a objetos y aprovecha características específicas de servidores MySQL modernos.
- **PDO (PHP Data Objects):** Es una capa de abstracción orientada a objetos. Su principal ventaja es que define una interfaz única para interactuar con múltiples motores de bases de datos (MySQL, PostgreSQL, SQLite). Si el proyecto cambia de motor, el código de las consultas base permanece prácticamente idéntico.

Ejemplo de conexión robusta utilizando PDO:

```
<?php
$dsn = "mysql:host=localhost;dbname=tienda_db;charset=utf8mb4";
$usuario = "root";
$password = "secreto123";

try {
    // Instanciación de la conexión con opciones de configuración segura
    $pdo = new PDO($dsn, $usuario, $password, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, // Activa el manejo de
    excepciones
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC // Retorna arreglos
    asociativos
    ]);
} catch (PDOException $e) {
```

```
// En producción, nunca se debe mostrar el mensaje crudo del error por
seguridad
die("Error crítico de conexión al servidor de datos.");
}
?>
```

La Inyección SQL (SQLi) es una de las vulnerabilidades más críticas en aplicaciones web, ya que ocurre cuando datos maliciosos provistos por un usuario son concatenados directamente dentro de una consulta SQL, permitiendo así al atacante alterar la lógica de la base de datos para extraer, modificar o borrar información protegida [4]. Por esta razón, para erradicar este riesgo de forma absoluta, el desarrollo moderno exige el uso obligatorio de Sentencias Preparadas (Prepared Statements) [2], [4].

Cuando se utiliza una sentencia preparada, el ciclo cambia drásticamente. En primer lugar, se produce una compilación separada, donde se envía la estructura de la consulta al servidor de la base de datos reemplazando los datos reales por marcadores de posición (parámetros :id o ?), de modo que el motor de la base de datos compila y optimiza la consulta antes de recibir los datos. Posteriormente, se realiza la inyección segura de datos, ya que estos se envían de manera aislada por un canal secundario, y el motor de la base de datos los trata estrictamente como cadenas literales o números, nunca como código ejecutable.

Ejemplo práctico de prevención de vulnerabilidades con PDO:

```
<?php
// CÓDIGO VULNERABLE (NUNCA HACER ESTO): Concatenación directa
// $input = $_POST['correo'];
// $sql = "SELECT * FROM usuarios WHERE email = '$input'";
// Si el input es: ' OR '1'='1 , el atacante salta la autenticación.

// CÓDIGO SEGURO: Usando Sentencias Preparadas
$inputCorreo = $_POST['correo'] ?? '';

// 1. Preparar la plantilla de la consulta en el servidor de la BD
$stmt = $pdo->prepare("SELECT id, nombre, password_hash FROM usuarios WHERE
email = :correo_usuario");

// 2. Vincular los parámetros de forma segura y ejecutar
$stmt->execute([
    ':correo_usuario' => $inputCorreo
]);

// 3. Obtener el resultado sanitizado
$usuarioEncontrado = $stmt->fetch();
?>
```

2.4 Comparativa Tecnológica y Criterios de Selección

La elección de una tecnología para el desarrollo del lado del servidor no debe responder a preferencias personales o modas temporales, sino a una evaluación meticulosa de las necesidades del negocio, la infraestructura disponible y el ciclo de vida del proyecto [3]. Un error en la selección del stack tecnológico puede derivar en problemas críticos de escalabilidad, sobrecostos de mantenimiento o vulnerabilidades de seguridad a mediano plazo [4].

A continuación, se presenta una matriz de selección basada en los criterios técnicos y organizacionales más importantes de la industria actual, contrastando el ecosistema de las dos tecnologías analizadas en este bloque (PHP y Perl) junto con las alternativas modernas del mercado (como Node.js, Python o Java):

Criterio de Selección	¿Qué evalúa?	PHP	Perl	Tecnologías Modernas
Rendimiento	Velocidad y manejo de usuarios simultáneos.	Alto. Muy optimizado en versiones 8.x con motor JIT.	Bajo/Medio. Limitado en webs de alto tráfico por su modelo CGI.	Excelente. Node.js lidera en conexiones simultáneas; Java en grandes empresas.
Seguridad	Protección contra hackeos (ej. Inyección SQL).	Alta. Posee herramientas modernas (PDO) para evitar fallos.	Alta. Es robusto, pero su sintaxis compleja facilita errores humanos.	Alta. Cuentan con frameworks modernos con seguridad automatizada.
Curva de Aprendizaje	Facilidad para aprender y escribir código.	Baja. Sintaxis sencilla, amigable para principiantes.	Muy Alta. Sintaxis compacta y difícil de leer ("código críptico").	Media. Varía según el lenguaje (Python es fácil; Java es más complejo).
Ecosistema y Comunidad	Soporte, librerías y documentación.	Gigante. Documentación oficial masiva y miles de librerías.	Maduro, pero Reducido. CPAN es excelente, pero hay poco soporte web actual.	Masivo. Comunidades enormes y herramientas de desarrollo ultra rápidas.
Disponibilidad de Talento	Facilidad para contratar programadores.	Abundante. Hay muchos desarrolladores y servidores económicos.	Escaso. Muy difícil encontrar programadores web en la actualidad.	Muy Abundante. Alta demanda y gran cantidad de profesionales.

Tabla 3. Comparativa tecnológica (PHP, Perl).

3. TEMA 2: DESARROLLO WEB CON JAVA Y JSP

3.1 Java para el Desarrollo Web: Plataforma y Herramientas

Java se mantiene como uno de los pilares fundamentales en el desarrollo de software de grado empresarial debido a su robustez, seguridad nativa y arquitectura fuertemente orientada a objetos [8]. Su principal ventaja competitiva radica en la Máquina Virtual de Java (JVM), la cual interpreta el código intermedio (bytecode) bajo el paradigma clásico Write Once, Run Anywhere (WORA); esto hace que las aplicaciones web backend sean completamente portables e independientes del sistema operativo del servidor subyacente [8], [9]. En el panorama actual, el desarrollo web en Java ha evolucionado desde los monolitos tradicionales hacia arquitecturas modulares y distribuidas, sustentadas por un ecosistema estandarizado y herramientas de automatización de alto rendimiento.

JDK vs. JRE: El entorno de ejecución e infraestructura de desarrollo de Java se organiza en capas de software bien definidas, cuyas especificaciones son dictadas por la documentación oficial de la plataforma [10].

Componente	¿Qué es?	Componentes Clave	Propósito en el Desarrollo Web
JDK (<i>Java Development Kit</i>)	Kit de desarrollo integral para el programador.	Compilador (javac), la JVM, herramientas de depuración y bibliotecas de la biblioteca estándar (<i>Core API</i>).	Obligatorio en desarrollo: Se encarga de procesar los archivos de código fuente (.java) y transformarlos en archivos binarios (.class) legibles por la máquina virtual antes del despliegue en el servidor.
JRE (<i>Java Runtime Environment</i>)	Entorno de ejecución requerido para servidores.	Máquina Virtual de Java (JVM) y los archivos mínimos de configuración de la plataforma.	Evolución tecnológica: A partir de Java 11 y consolidado en versiones de soporte a largo plazo (LTS) como JDK 21, el JRE independiente ya no se distribuye oficialmente, integrándose directamente dentro del JDK para unificar el ciclo [10].

Tabla 4. Capas de software (JDK vs JRE).

Jakarta EE (Antes Java EE): Originalmente conocido como Java EE bajo la tutela de Oracle, la gobernanza de la plataforma empresarial fue transferida a la Eclipse Foundation en 2017, adoptando el estándar abierto Jakarta EE [11]. En lugar de ser un único framework, Jakarta EE es una colección de especificaciones abstractas que definen cómo deben interactuar los componentes web corporativos, permitiendo el desarrollo de aplicaciones modulares y altamente escalables.

Las especificaciones centrales para el backend web se estructuran en las siguientes capas de servicio [11]:

Especificación	Propósito Técnico en el Servidor	Referencia Oficial
Jakarta Servlet	Gestiona el ciclo de vida de las peticiones HTTP y las respuestas del lado del servidor de forma asíncrona.	Specification v6.1

Especificación	Propósito Técnico en el Servidor	Referencia Oficial
Jakarta Server Pages (JSP) & EL	Permite embeber lógica dinámica dentro de vistas basadas en texto tradicional (HTML/XML).	Specification v4.0
Jakarta RESTful Web Services (JAX-RS)	Define las anotaciones y componentes estructurales para el diseño y consumo de APIs REST.	Platform Spec v11
Jakarta Persistence (JPA)	Proporciona una interfaz de mapeo objeto-relacional (ORM) para la persistencia transparente de datos en RDBMS.	Platform Spec v11

Tabla 5. Especificaciones centrales para el backend web.

Beneficios Empresariales: El cumplimiento de estas especificaciones garantiza una arquitectura estandarizada que evita el acoplamiento a un proveedor (vendor lock-in), asegurando que el código sea compatible con cualquier servidor certificado.

Herramientas de Construcción (Build Tools): Los desarrollos web empresariales demandan la automatización de tareas críticas como la gestión de dependencias, la compilación cruzada, las pruebas de software y el empaquetado (en archivos .war o .jar).

Herramienta	Archivo Central	Enfoque y Ventajas Operativas	Escenario de Uso
Apache Maven	pom.xml (<i>Project Object Model</i>)	Basado en la convención sobre la configuración: Impone una estructura de directorios estricta y predecible. Automatiza la descarga y resolución de dependencias desde repositorios globales de manera jerárquica [12].	Proyectos corporativos estándar y configuraciones iniciales del ecosistema Spring.
Gradle	build.gradle (<i>Groovy / Kotlin DSL</i>)	Basado en scripts programables: Ofrece un rendimiento superior gracias a su motor de compilación incremental y caché de compilación, permitiendo una personalización total del pipeline de despliegue [13].	Microservicios modernos, proyectos multi-módulo de gran escala y entornos híbridos.

Tabla 6. Herramientas de construcción.

Contenedores y Servidores de Aplicaciones: Para ejecutar aplicaciones Java en la web se requiere un software intermedio (middleware) que implemente las especificaciones de Jakarta EE y gestione el ciclo de vida de las peticiones de red.

Servidor de Aplicaciones	Tipo de Entorno	Cobertura de Especificaciones	Perfil Técnico de Uso
Apache Tomcat	Contenedor Web (<i>Web Container</i>)	Implementa exclusivamente el perfil web básico: Jakarta Servlet, JSP y Expression Language [14].	Es la solución más popular de la industria. Destaca por ser ligero, veloz y es el motor embebido por defecto en entornos modernos de Spring Boot [14].
WildFly (Red Hat)	Servidor Completo (<i>Full Application Server</i>)	Implementa la totalidad del stack y perfiles avanzados de la especificación Jakarta EE (JPA, JAX-RS, mensajería, transacciones distribuidas) [15].	Diseñado para grandes sistemas empresariales unificados que requieren múltiples servicios integrados nativamente en el mismo servidor.
Eclipse Jetty	Contenedor Web (<i>Embebible</i>)	Enfocado en especificaciones web básicas, optimizando al máximo el uso de memoria RAM [16].	Diseñado para alta escalabilidad horizontal. Su arquitectura permite integrarlo directamente dentro del código ejecutable del microservicio, siendo ideal para despliegues en la nube y contenedores (Docker) [16].

Tabla 7. Servidores de aplicaciones.

3.2 Java Servlets

Un Jakarta Servlet es una clase Java que extiende las capacidades de un servidor web para responder a peticiones dinámicas basadas en el protocolo HTTP [17]. Actúa como el intermediario directo entre el cliente (navegador) y la lógica de negocio de la aplicación. Dentro de la arquitectura de la plataforma, los Servlets no poseen un método `main()`; en su lugar, se ejecutan dentro de un contenedor web (como Apache Tomcat) que se encarga de administrar por completo su ciclo de vida, la asignación de hilos de ejecución (threads) y la seguridad de la red [14].

En el ciclo de vida del Servlet el contenedor web gestiona rigurosamente el ciclo de vida de un Servlet a través de tres fases operativas principales definidas por la especificación oficial [17]:

- **Inicialización (`init()`):** Cuando se realiza la primera petición al Servlet (o al arrancar el servidor si está configurado), el contenedor carga la clase en memoria y ejecuta el método `init()`. Este método se ejecuta una sola vez en toda la existencia del Servlet y se utiliza para configuraciones iniciales pesadas (como abrir conexiones genéricas o cargar archivos de propiedades) [17].
- **Servicio (`service()`):** Por cada petición HTTP entrante, el contenedor asigna un hilo (thread) dedicado y llama al método `service()`. Este método analiza el tipo de petición (GET, POST, PUT, DELETE) y desvía el flujo automáticamente hacia los métodos especializados correspondientes (`doGet()`, `doPost()`, etc.) [17].
- **Destrucción (`destroy()`):** Cuando el contenedor decide dar de baja el Servlet (debido a falta de memoria o al apagar el servidor), se invoca el método `destroy()`. También se ejecuta una sola vez y su función es liberar recursos activos, cerrar hilos secundarios y guardar estados persistentes.

El mapeo de URL con `@WebServlet` en las versiones antiguas de Java EE, la asociación de una URL con un Servlet específico requería extensas configuraciones en un archivo XML llamado `web.xml`. En el estándar actual de Jakarta EE, este proceso se simplificó mediante el uso de la anotación nativa `@WebServlet`, la cual realiza el mapeo de forma declarativa directamente sobre el código fuente [11], [17].

Ejemplo:

```
package com.sistema.web;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;

// Mapeo declarativo: cualquier petición a http://localhost:8080/app/usuarios
// será dirigida aquí
@WebServlet(name = "UsuarioServlet", urlPatterns = {"/usuarios", "/clientes/*"})
public class UsuarioServlet extends HttpServlet {
    // La clase debe heredar de HttpServlet para obtener el comportamiento web
}
```

Manejo de peticiones, respuestas y sesiones: El núcleo de la interacción web se basa en dos objetos que el contenedor inyecta automáticamente en los métodos de servicio: `HttpServletRequest` (la petición del cliente) y `HttpServletResponse` (la respuesta del servidor) [17].

Mecanismo Técnico	Componente / API	Propósito y Funcionamiento en el Servidor
Manejo de GET	Método <code>doGet()</code>	Se utiliza para solicitar datos del servidor. Los parámetros viajan visibles en la URL y se extraen mediante el método <code>request.getParameter("llave")</code> [17].
Manejo de POST	Método <code>doPost()</code>	Utilizado para enviar información sensible o masiva (ej. formularios). Los datos viajan ocultos en el cuerpo de la petición HTTP y se leen de igual forma sin exponerse en la red.
Persistencia de Estado	Interfaz <code>HttpSession</code>	HTTP no tiene estado (<i>stateless</i>). Al invocar <code>request.getSession()</code> , el contenedor crea una sesión en la memoria del servidor y genera un identificador único (JSESSIONID) asignado al usuario [17].
Rastreo del Cliente	Cookies Nativas	El JSESSIONID se envía automáticamente al navegador mediante una cookie oculta. En cada nueva interacción, el navegador devuelve la cookie, permitiendo al Servlet identificar al usuario y recuperar su información en el backend [17].

Tabla 7. Mecanismo y componente api.

A continuación, se detalla un ejemplo práctico integrando estos componentes:

```
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;
import java.io.PrintWriter;
```



```

public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        // 1. Capturar parámetros de la petición HTTP POST
        String txtUsuario = request.getParameter("user");

        // 2. Gestión de Sesión
        HttpSession session = request.getSession(true); // Crea una sesión si no
existe
        session.setAttribute("usuarioActivo", txtUsuario); // Almacena el dato
en el servidor

        // 3. Construcción de la Respuesta HTTP
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            out.println("<html><body>");
            out.println("<h2>Sesión iniciada con éxito para: " + txtUsuario +
"</h2>");
            out.println("</body></html>");
        }
    }
}

```

Filtros y listeners: Para construir arquitecturas web modulares y limpias, Jakarta EE delega las tareas transversales e interceptación a dos componentes especializados que actúan antes o después de la ejecución del Servlet:

Componente	Anotación	Mecanismo de Acción	Casos de Uso Comunes
Filtros (Filters)	@WebFilter	Interceptan las peticiones y respuestas en cadena antes de que lleguen al Servlet final. Tienen la capacidad de bloquear la solicitud o alterar las cabeceras HTTP.	- Control de autenticación (verificar si el usuario inició sesión). - Compresión de datos (GZIP). - Sanitización de entradas contra ataques informáticos.
Listeners	@WebListener	Son componentes pasivos basados en el patrón de diseño <i>Observer</i> . Reaccionan de forma automática ante eventos globales del contenedor web.	- Conectar a la base de datos al iniciar la aplicación (ServletContextListener). - Registrar cuántos usuarios hay activos destruyendo o creando sesiones (HttpSessionListener).

Tabla 8. Filtros y listeners.

3.3 JavaServer Pages (JSP)

Jakarta Server Pages (JSP) es una tecnología que permite a los desarrolladores crear vistas web dinámicas combinando HTML estático con lógica de programación Java [18]. Conceptualmente, JSP funciona de manera inversa a un Servlet: mientras que en un Servlet se escribe código Java que imprime texto HTML, en un JSP se escribe texto HTML que contiene etiquetas especiales de Java.

El secreto de JSP: Un archivo .jsp no es interpretado directamente por el servidor en cada petición. La primera vez que el cliente solicita la página, el contenedor web traduce automáticamente el archivo JSP en un código fuente de Servlet de Java y luego lo compila en un archivo .class [18]. Por lo tanto, un JSP es, internamente, un Servlet optimizado para la capa de presentación.

La especificación oficial define diferentes etiquetas para inyectar comportamiento dinámico dentro del documento HTML:

Elemento Sintáctico	Sintaxis	Propósito Técnico	Ejemplo Práctico
Directivas	<code><%@ ... %></code>	Definen instrucciones globales para el contenedor web sobre cómo se debe procesar y traducir la página entera [18].	<pre><%@ page contentType="text/html" %> Dragon Ball import="java.util.List" %></pre>
Scriptlets	<code><% ... %></code>	Permiten insertar bloques de código fuente Java crudo que se ejecutarán dentro del método de servicio de la página. <i>(Nota: Su uso actual está desaconsejado porque ensucia la vista).</i>	<pre><% int contador = 10; if(contador > 5) { %></pre>
Expresiones	<code><%= ... %></code>	Evalúan una sola variable o instrucción en Java, la convierten en texto (String) y la imprimen directamente en el HTML final [18]. No llevan punto y coma.	Hoy es: <code><%= new java.util.Date() %></code>

Tabla 9. Etiquetas para comportamiento dinámico.

EL (Expression Language) y JSTL: Para evitar el uso de código Java crudo (Scriptlets) dentro del HTML y mantener una arquitectura limpia, las versiones modernas de Jakarta EE exigen combinar dos herramientas estándar [11], [18].

- **Expression Language (EL):** Es una sintaxis simplificada que permite acceder de forma elegante a los datos almacenados en los objetos y ámbitos de Java (request, session, application) usando la nomenclatura `${objeto.propiedad}` [18].
- **JSTL (Jakarta Standard Tag Library):** Es una colección de etiquetas listas para usar (similares a HTML) que resuelven problemas comunes de lógica visual como bucles, condicionales y formateo de datos, sin escribir una sola línea de Java [18].

Ejemplo:

```
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
```

```

<html>
<body>
  <h2>Lista de Productos Disponibles</h2>
  <ul>
    <c:forEach var="producto" items="${listaProductos}">
      <li>${producto.nombre} - Precio: ${producto.precio}</li>
    </c:forEach>
  </ul>
</body>
</html>

```

Integración con Servlets: El Patrón MVC (Model 2): En aplicaciones reales, nunca se debe permitir que un archivo JSP maneje la lógica de negocio o acceda directamente a las bases de datos. La arquitectura estándar de la industria dicta el uso del Patrón MVC (conocido históricamente en Java como Arquitectura Model 2), donde los Servlets y los JSP cooperan de forma estrecha [17].

- **Controlador (Servlet):** Es el punto de entrada único. Recibe la petición del cliente, extrae los parámetros del formulario, invoca los servicios del backend y guarda los resultados (el Modelo) en el objeto `HttpServletRequest` [17].
- **Modelo (JavaBeans / Clases Java):** Representa los datos puros de la aplicación y las reglas del negocio (ej. una clase Cliente o un servicio de base de datos).
- **Vista (JSP):** Recibe los datos ya procesados desde el controlador mediante un despacho interno (`RequestDispatcher`) y se limita exclusivamente a renderizar el HTML final utilizando EL y JSTL [18].

A continuación, se muestra cómo se conecta este flujo mediante código:

1. El Controlador (Servlet que procesa y redirige)

```

@WebServlet("/listar-items")
public class ItemServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        // 1. Crear o recuperar el Modelo (Simulación de datos)
        List<String> items = Arrays.asList("Laptop", "Teclado", "Monitor");

        // 2. Adjuntar el Modelo a la petición HTTP
        request.setAttribute("misItems", items);

        // 3. Despachar (redirección interna en el servidor) hacia la Vista JSP
        RequestDispatcher dispatcher =
request.getRequestDispatcher("/vistas/lista.jsp");
        dispatcher.forward(request, response); // Entrega el control al JSP
    }
}

```

2. La Vista (Archivo /vistas/lista.jsp que solo dibuja)

```
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
<!DOCTYPE html>
<html>
<head><title>Inventario</title></head>
<body>
  <h1>Items del Sistema</h1>
  <ul>
    <c:forEach var="item" items="${misItems}">
      <li><c:out value="${item}" /></li>
    </c:forEach>
  </ul>
</body>
</html>
```

3.4 Introducción a Spring Boot para Desarrollo Web

Spring Boot es una extensión del ecosistema de Spring Framework diseñada para simplificar radicalmente el desarrollo de aplicaciones listas para producción [19]. Tradicionalmente, configurar un proyecto empresarial en Java requería escribir cientos de líneas en archivos XML o clases de configuración pesadas. Spring Boot soluciona esto introduciendo dos pilares fundamentales [19], [20]:

- **Autoconfiguración (Opinionated Configuration):** El framework "asume" lo que la aplicación necesita basándose en las librerías añadidas al proyecto (ej. si detecta el conector de MySQL en las dependencias, configura automáticamente la conexión a la base de datos) [19].
- **Servidor Embebido:** No es necesario empaquetar un archivo .war y desplegarlo manualmente en un servidor externo. Spring Boot incluye un servidor Apache Tomcat embebido por defecto, compilando la aplicación en un archivo .jar ejecutable e independiente [14], [19].

Estructura de un Proyecto Spring Boot: A través de herramientas de automatización como Maven o Gradle [8], [9], Spring Boot impone una arquitectura estandarizada y limpia [19]:

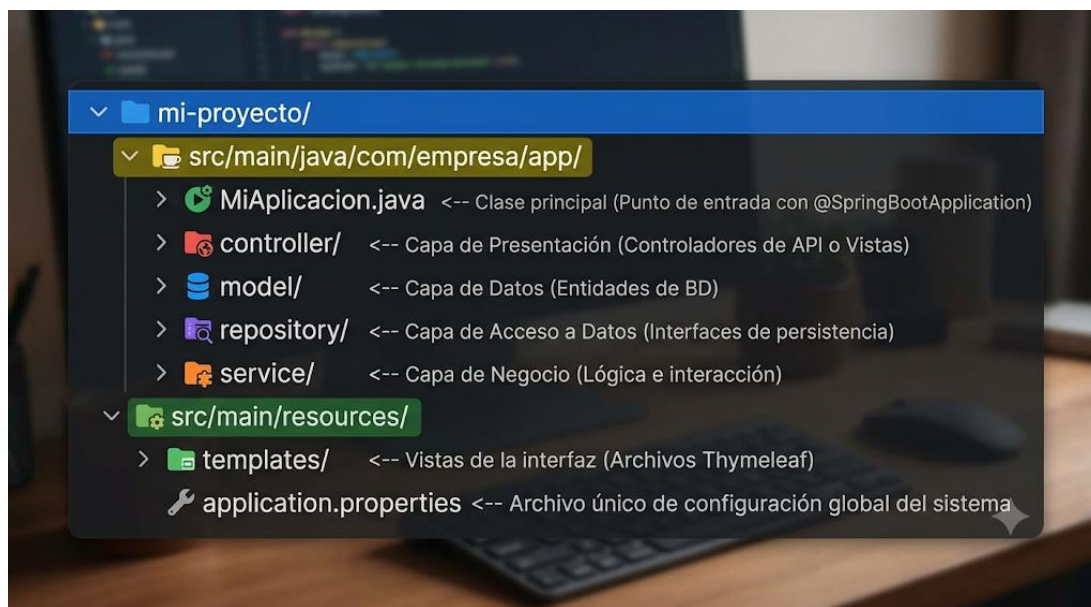


Fig. 2. Estructura de un proyecto Backend (Springboot).

Controladores: @Controller vs. @RestController: El módulo Spring MVC gestiona el flujo de las peticiones HTTP mediante la capa de control. Spring Framework divide los controladores en dos tipos especializados según el propósito de la aplicación [19], [21]:

Característica	@Controller	@RestController
Arquitectura	Aplicaciones Web Tradicionales (Monolíticas).	Arquitecturas desacopladas y APIs REST [20].
Retorno por defecto	Devuelve el nombre de una vista estática o plantilla (HTML) que debe ser renderizada en el servidor [21].	Devuelve datos puros (objetos Java) que se transforman automáticamente a formato JSON [19].
Motor de Vistas	Integra motores como Thymeleaf para inyectar datos directamente sobre las etiquetas HTML.	No maneja interfaces; sirve datos a aplicaciones móviles o frontends modernos (React, Angular).

Tabla 10. @Controller vs @RestController, características.

Ejemplo:

```
// Ejemplo de un RestController moderno para una API REST
@RestController
@RequestMapping("/api/productos")
public class ProductoRestController {

    @GetMapping("/{id}")
    public ResponseEntity<String> obtenerProducto(@PathVariable Long id) {
        // Retorna un JSON puro de forma directa al cliente
        return ResponseEntity.ok("{\"id\": " + id + ", \"nombre\": \"Laptop\"}");
    }
}
```

Persistencia con Spring Data JPA: Spring Data JPA lleva la abstracción de bases de datos al nivel más alto posible, encapsulando las especificaciones tradicionales de Jakarta JPA [19].

En lugar de escribir consultas SQL complejas o gestionar transacciones de forma manual, el desarrollador define una Entidad (una clase Java mapeada a una tabla de base de datos con la anotación @Entity) y hereda la interfaz JpaRepository. Automáticamente, Spring implementa en tiempo de ejecución todas las operaciones CRUD (Crear, Leer, Actualizar, Borrar) y consultas avanzadas sin necesidad de escribir código ejecutable [21]:

```
// Interfaz que maneja de forma automática la persistencia en la base de datos
@Repository
public interface ProductoRepository extends JpaRepository<Producto, Long> {
    // Spring genera la consulta automáticamente interpretando el nombre del método
    List<Producto> findByNombreContaining(String termino);
}
```

Seguridad con Spring Security: Spring Security es el estándar de facto para la protección de aplicaciones basadas en Spring, implementando mecanismos robustos contra las vulnerabilidades web más comunes (como secuestros de sesión, ataques por fuerza bruta o falsificación de solicitudes entre sitios CSRF) [20], [19]. Funciona mediante una cadena de filtros interceptores (Security Filter Chain) que procesan la petición del cliente antes de permitir el acceso a los controladores [21], delegando dos funciones críticas:

- **Autenticación:** Verifica la identidad del usuario (¿Quién eres?). Admite desde esquemas de formularios tradicionales en memoria o bases de datos relacionales, hasta arquitecturas modernas basadas en tokens JWT (JSON Web Tokens) u OAuth2.
- **Autorización:** Define las restricciones de acceso según roles o permisos (¿Qué tienes permitido hacer?). Permite proteger rutas web específicas de manera declarativa desde archivos de configuración o anotaciones en los métodos de negocio.

```
// Ejemplo simplificado de configuración de seguridad (Spring Boot 3.x)
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/publico/**").permitAll() // Rutas de libre
                .requestMatchers("/api/admin/**").hasRole("ADMIN") //
                .anyRequest().authenticated() // Todo lo demás requiere login
            )
            .formLogin(Customizer.withDefaults()); // Activa el formulario de
            return http.build();
    }
}
```

4. TEMA 3: DESARROLLO WEB CON ASP.NET

4.1 La Plataforma .NET y ASP.NET

El ecosistema de desarrollo de Microsoft ha experimentado una de las transformaciones arquitectónicas más radicales de la industria, evolucionando hacia un modelo de código abierto, modular y de alto rendimiento adaptado a las necesidades de la computación en la nube [22], [23]. Para comprender el diseño actual de ASP.NET Core, es indispensable analizar la unificación de su plataforma y los componentes de bajo nivel que sustentan su infraestructura de ejecución [23].

La transición histórica desde las arquitecturas propietarias hacia el entorno moderno se divide en tres etapas fundamentales bien documentadas:

Era Tecnológica	Plataforma Principal	Características y Limitaciones Técnicas	Impacto en el Desarrollo Web
Era Tradicional (Hasta 2016)	.NET Framework	Ecosistema propietario y de código cerrado. Diseñado con una fuerte dependencia monolítica, estaba vinculado exclusivamente al sistema operativo Windows y al servidor web IIS (<i>Internet Information Services</i>) [24].	Las aplicaciones se construían con ASP.NET clásico (Web Forms / MVC primitivo). Eran plataformas robustas pero rígidas, pesadas y costosas de escalar en entornos de nube [22].
Era de Transición (2016 - 2020)	.NET Core (1.0 a 3.1)	Reinvención total desde los cimientos. Se transformó en un entorno 100% de código abierto (Open Source) y multiplataforma , capaz de ejecutarse de forma nativa en Linux, macOS y Windows [25].	Nace ASP.NET Core , un framework optimizado, ligero y modular basado en un pipeline de ejecución por capas, ideal para contenedores Docker y microservicios [25], [26].
Era Moderna (Actualidad)	.NET 5 en adelante	Microsoft eliminó la palabra "Core" para unificar todas sus líneas de desarrollo (móvil, web, escritorio) en un único motor de ejecución global, consolidando la madurez de la plataforma [22], [23].	ASP.NET Core en las versiones actuales se posiciona en los primeros puestos de los benchmarks de la industria (como TechEmpower) gracias a su velocidad y su enfoque nativo para la nube (<i>Cloud-Native</i>) [23].

Tabla 11. Transición tecnológica de .NET.

La estabilidad, portabilidad y el rendimiento extremo de las aplicaciones web en este ecosistema se sostienen sobre dos pilares de infraestructura administrados por la plataforma [23]:

- **CLR (Common Language Runtime):** Es el entorno de ejecución de lenguaje común (el motor de la plataforma), análogo a la máquina virtual de otros ecosistemas [26]. Cuando un proyecto en C# se compila, se traduce a un código binario intermedio denominado IL (Intermediate Language). En tiempo de ejecución, el CLR toma este código IL y, mediante un compilador JIT (Just-In-Time), lo transforma directamente en instrucciones nativas de la arquitectura del procesador del servidor (X64, ARM64, etc.)

[22]. Además, el CLR automatiza la gestión de memoria mediante el Garbage Collector, administra los hilos de red y previene fallos de seguridad en la ejecución [23].

- **BCL (Base Class Library):** Es la biblioteca de clases base unificada. Proporciona una colección estandarizada, integrada y optimizada de APIs reutilizables que resuelven los servicios esenciales de bajo y alto nivel de la aplicación, tales como la manipulación segura de cadenas, colecciones avanzadas (diccionarios, listas), criptografía, operaciones de entrada/salida de archivos y gestión de protocolos de red mediante clientes HTTP [23].

El Ecosistema NuGet: NuGet es el gestor de paquetes y dependencias oficial para la plataforma .NET [23]. Su función primordial es centralizar y automatizar el ciclo de vida de las librerías de software de terceros dentro de los proyectos de desarrollo [22].

A través de un archivo de definición de proyecto basado en XML (con extensión .csproj), NuGet registra las referencias exactas y restricciones de versión requeridas por la aplicación [23]. Esto simplifica drásticamente el flujo de trabajo, ya que permite que los servidores de integración continua (CI/CD) o cualquier desarrollador del equipo puedan restaurar, descargar e integrar automáticamente todo el árbol de dependencias necesarias con una sola instrucción, garantizando la repetibilidad del entorno tanto en desarrollo como en producción [22], [23].

4.2 ASP.NET Core: Estructura de un Proyecto Web

Un proyecto web moderno en ASP.NET Core destaca por ser minimalista, ligero y completamente modular [23]. A diferencia de las arquitecturas antiguas cargadas de archivos de configuración complejos, el desarrollo actual concentra el arranque, el aprovisionamiento de servicios y el flujo de control en una estructura limpia de archivos esenciales [22], [27].

Al crear una aplicación web en ASP.NET Core, el andamiaje del proyecto se organiza en torno a los siguientes componentes clave [27]:

- **Program.cs:** Es el punto de entrada único de la aplicación [22]. Contiene el código de inicio que configura el servidor web (Kestrel), registra los servicios en el contenedor de inversión de control y define el pipeline de procesamiento de solicitudes HTTP [27].
- **appsettings.json:** Archivo de configuración central en formato JSON. Se utiliza para almacenar parámetros de la aplicación que pueden cambiar según el entorno, tales como cadenas de conexión a bases de datos, claves de APIs de terceros o niveles de registro de logs [27].
- **MiProyecto.csproj:** Archivo XML del proyecto que gestiona las herramientas de compilación, la versión de destino de .NET y los paquetes NuGet instalados [27].

Middleware y Pipeline de Solicitudes: El procesamiento de una petición HTTP en ASP.NET Core se basa en un concepto llamado Pipeline de Middleware [27]. Un middleware es un componente de software (un bloque de código aislado) que se ensambla en la tubería de la aplicación para manejar las solicitudes y respuestas entrantes [23].

Cada middleware en el pipeline tiene una responsabilidad única y secuencial:

1. Recibe la solicitud HTTP del componente anterior.
2. Ejecuta su lógica interna (ej. verificar si el usuario está autenticado).

- Decide si pasa la solicitud al siguiente middleware en la cadena mediante la instrucción `next()` o si rompe el circuito (short-circuit) devolviendo una respuesta inmediata (ej. denegar el acceso).

El orden en el que se registran los middlewares dentro del archivo `Program.cs` es estrictamente crucial, ya que define la lógica secuencial de la seguridad y el rendimiento de la red, por ejemplo:

```
// Fragmento de Program.cs: Configuración del Pipeline de Middleware
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// 1. Middleware de manejo de excepciones globales
if (!app.Environment.IsDevelopment()) {
    app.UseExceptionHandler("/Home/Error");
}

// 2. Servir archivos estáticos (HTML, CSS, imágenes) de forma eficiente
app.UseStaticFiles();

// 3. Middleware de enrutamiento (mapeo de URLs)
app.UseRouting();

// 4. Middleware de Autenticación y Autorización (Seguridad)
app.UseAuthentication();
app.UseAuthorization();

// 5. Middleware final: Enrutar la petición hacia los controladores
app.MapControllerRoute(name: "default", pattern:
"{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Inyección de Dependencias (DI): ASP.NET Core incluye un contenedor de Inyección de Dependencias (DI) nativo y de primera clase, lo que significa que el framework está diseñado desde sus cimientos bajo el principio de Inversión de Control [22], [23]. La DI permite construir aplicaciones desacopladas, modulares y fáciles de probar de forma unitaria, eliminando la necesidad de que las clases creen manualmente sus propias dependencias utilizando el operador `new` [27].

Los servicios se registran en el contenedor dentro de `Program.cs` especificando uno de los tres ciclos de vida (lifetimes) según la necesidad del recurso:

Ciclo de Vida	Método de Registro	Comportamiento en el Servidor	Caso de Uso Común
Transient	<code>builder.Services.AddTransient<IObjeto, Objeto>();</code>	Se crea una nueva instancia cada vez que se solicita el servicio al contenedor, incluso dentro de la misma petición HTTP [27].	Servicios ligeros sin estado (<i>stateless</i>) o cálculos matemáticos simples.

Ciclo de Vida	Método de Registro	Comportamiento en el Servidor	Caso de Uso Común
Scoped	<code>builder.Services.AddScoped<IObjeto, Objeto>();</code>	Se crea una sola instancia por cada petición HTTP . La instancia se comparte entre todos los componentes que la requieran durante esa solicitud y se destruye al finalizarla [22].	El contexto de la base de datos (DbContext), repositorios o gestores de usuarios activos [27].
Singleton	<code>builder.Services.AddSingleton<IObjeto, Objeto>();</code>	Se crea una única instancia la primera vez que se solicita y permanece activa en la memoria del servidor durante todo el ciclo de vida de la aplicación web [27].	Caché en memoria del servidor, lectores de configuración global o servicios de log compartidos.

Tabla 12. Ciclos de vida en DI.

Configuración y Entornos de Ejecución: El framework posee un sistema de configuración dinámico que permite adaptar el comportamiento de la aplicación web de forma automática según el entorno donde se esté ejecutando (comúnmente Development para desarrollo local y Production para el servidor en vivo) [22], [27].

1. Esto se logra mediante la jerarquía de lectura del archivo appsettings.json:
2. El servidor lee primero el archivo base appsettings.json.
3. Inmediatamente después, si detecta la variable de entorno del sistema ASPNETCORE_ENVIRONMENT=Development, busca y lee el archivo específico appsettings.Development.json.
4. Los valores definidos en el archivo del entorno específico sobrescriben automáticamente a los del archivo base.

Ejemplos:

```
// Ejemplo de appsettings.Development.json (Configuración detallada para
programadores)
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Database=DevDb;Trusted_Connection=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Debug", // Muestra errores detallados en la consola de
desarrollo
    }
  }
}
```

```

    "Microsoft.AspNetCore": "Information"
  }
}
}

// Ejemplo de appsettings.Production.json (Configuración optimizada y segura)
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=ServidorProduccion;Database=ProdDb;User
Id=Admin;Password=ClaveSegura;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Error" // Solo registra errores críticos para proteger el
rendimiento
    }
  }
}
}

```

4.3 Razor Pages y Razor Views

El motor de plantillas Razor es la tecnología de renderizado del lado del servidor nativa de ASP.NET Core [5]. Su propósito es permitir a los desarrolladores mezclar código C# estructurado con marcado HTML tradicional de una manera fluida, intuitiva y con un alto rendimiento computacional [22], [27].

Sintaxis @ (Razor Syntax)

La característica fundamental de Razor es el uso del carácter @ como un delimitador único [27]. A diferencia de otros lenguajes de plantillas antiguos que requieren etiquetas de apertura y cierre complejas, el motor de Razor realiza una transición limpia entre HTML y C# interpretando el símbolo @ de dos formas principales [22]:

- **Expresiones implícitas y explícitas:** Permiten renderizar valores directamente en el flujo de texto HTML (ej. `<span>@DateTime.Now.Year</span>`).
- **Bloques de código:** Permiten escribir lógica de control estructurada en C# (como condicionales o bucles) que alteran el HTML final en tiempo de ejecución.

Ejemplo:

```

@if (Model.Productos.Any()) {
  <ul>
    @foreach (var prod in Model.Productos) {
      <li>@prod.Nombre - Precio: $@prod.Precio</li>
    }
  </ul>
} else {
  <p>No hay productos disponibles en este momento.</p>
}

```

Arquitecturas de Presentación: MVC vs. Razor Pages

ASP.NET Core ofrece dos enfoques integrados para organizar la lógica de la interfaz de usuario en el servidor, adaptándose a la escala del proyecto:

Característica	ASP.NET Core MVC (Model-View-Controller)	ASP.NET Core Razor Pages
Enfoque de Diseño	Orientado a la arquitectura general. Separa estrictamente la aplicación en tres componentes globales: Modelos, Vistas y Controladores [22].	Orientado a las páginas. Agrupa de forma lógica todo lo relacionado con una sola pantalla del sistema dentro de una estructura compacta [28].
Patrón de Arquitectura	MVC tradicional. Un controlador central (HomeController.cs) intercepta las peticiones y decide qué vista (Index.cshtml) renderizar [23].	MVVM / Page-Model. Utiliza un modelo de página dedicado acoplado directamente a la vista, eliminando la necesidad de controladores externos [28].
Estructura de Archivos	Los archivos están dispersos en múltiples carpetas (/Controllers, /Views/Home, /Models) [22].	Utiliza una pareja de archivos co-localizados: Index.cshtml (la interfaz) e Index.cshtml.cs (el código por detrás o <i>code-behind</i>) [28].
Escenario Ideal	Sistemas masivos con flujos de trabajo hiper-complejos y múltiples vistas compartiendo lógica [22].	Recomendado por Microsoft para aplicaciones web estándar orientadas a formularios, flujos CRUD rápidos y portales de contenido [22].

Tabla 13. Enfoques de ASP.NET Core.

Tag Helpers

Los Tag Helpers son una característica moderna de ASP.NET Core que permite al código del servidor participar en la creación y el renderizado de elementos HTML comunes en las vistas [22], [27].

A diferencia de los antiguos HTML Helpers de .NET Framework (que utilizaban sintaxis de código Java/C# intrusiva), los Tag Helpers parecen etiquetas y atributos HTML estándar, lo que facilita enormemente la legibilidad del código y la colaboración con diseñadores frontend [23].

Ejemplo:

```
<a class="btn btn-primary" asp-controller="Inventario" asp-action="Detalles"
asp-route-id="5">
    Ver Producto
</a>

<a class="btn btn-primary" href="/Inventario/Detalles/5">
    Ver Producto
</a>
```

Ventaja de los Tag Helpers: Generan URLs dinámicas y amigables de forma automática basadas en el sistema de enrutamiento de la aplicación, y proveen protección automática contra ataques de falsificación de peticiones en los formularios (anti-forgery tokens) [22], [27].

Partial Views (Vistas Parciales)

Una Vista Parcial es un archivo Razor (.cshtml) especializado que se utiliza para renderizar una sección fragmentada de HTML dentro de otra vista más grande [22], [27].

Su propósito central es promover la reutilización de código y la modularidad de la interfaz de usuario bajo el principio DRY (Don't Repeat Yourself). No contienen directivas globales como el layout de la página ni inicializaciones pesadas; simplemente reciben un modelo de datos específico y dibujan un componente visual reutilizable [27].

Ejemplo:

```
<div class="sidebar">
    <partial name="_MenuLateralPartial" model="Model.OpcionesMenu" />
</div>
```

4.4 Formularios, Validación y Seguridad en ASP.NET

El manejo seguro de la información introducida por los usuarios constituye una de las prioridades críticas de ASP.NET Core. El framework implementa una arquitectura automatizada por capas que se encarga de interceptar los datos, verificar su integridad e impedir la ejecución de código malicioso o accesos no autorizados antes de que toquen la lógica del negocio [27], [29].

Data Annotations y Validación de Modelos

El procesamiento de formularios en ASP.NET Core se fundamenta en un mecanismo gemelo: el Model Binding (Aoplamiento de Modelos) y las Data Annotations (Anotaciones de Datos).

- **Model Binding:** El framework analiza de forma automática la petición HTTP entrante (datos de formularios, parámetros de URL o el cuerpo de un JSON) y mapea esos valores directamente hacia las propiedades de un objeto de C# (un Modelo) [27].
- **Data Annotations:** Son atributos declarativos (metadatos) que se anteponen directamente sobre las propiedades del modelo para imponer reglas de negocio, formatos y restricciones de seguridad estrictas [27].

Cuando los datos pasan por el pipeline, el motor evalúa estas anotaciones y registra el resultado en la propiedad global ModelState.IsValid [30]. Si alguna regla se rompe, el servidor detiene el procesamiento, bloquea el guardado de datos y retorna los mensajes de error configurados [22].

Ejemplo:

```
using System.ComponentModel.DataAnnotations;

public class RegistroUsuarioViewModel {
    [Required(ErrorMessage = "El nombre de usuario es obligatorio.")]
    [StringLength(20, MinimumLength = 4, ErrorMessage = "Debe tener entre 4 y 20 caracteres.")]
    public string NombreUsuario { get; set; } = string.Empty;

    [Required(ErrorMessage = "El correo electrónico no puede estar vacío.")]
    [EmailAddress(ErrorMessage = "El formato del correo electrónico no es válido.")]
    public string CorreoElectronico { get; set; } = string.Empty;
}
```

```

public string Correo { get; set; } = string.Empty;

[Required]
[DataType(DataType.Password)]
public string Password { get; set; } = string.Empty;
}

```

La validación se evalúa de forma estricta en el controlador o en el PageModel antes de realizar cualquier acción en el backend:

```

[HttpPost]
public IActionResult Registrar(RegistroUsuarioViewModel modelo) {
    // Verificación obligatoria en el servidor
    if (!ModelState.IsValid) {
        return View(modelo); // Retorna los errores de validación a la interfaz
    }
    // Si es válido, continúa con la lógica de persistencia...
    return RedirectToAction("Index");
}

```

Token Anti-Forgery (Prevención de CSRF)

El ataque de Falsificación de Solicitudes entre Sitios (CSRF) ocurre cuando un sitio web malicioso logra que el navegador de un usuario autenticado envíe de forma involuntaria una petición HTTP legítima hacia una aplicación vulnerable donde el usuario ya tenía una sesión activa [29].

ASP.NET Core mitiga este riesgo de raíz utilizando el sistema Antiforgery Token basado en el patrón de doble cookie:

1. Cuando el servidor renderiza un formulario protegido, genera automáticamente dos elementos criptográficos: una cookie de seguridad oculta en el navegador y un campo de formulario de texto invisible que contiene un token único (`__RequestVerificationToken`) [27].
2. Al enviar el formulario, ambos componentes viajan al servidor. El middleware de validación intercepta la petición y compara criptográficamente el token del formulario con el valor de la cookie [29]. Si no coinciden o el campo falta, la solicitud es rechazada de inmediato con un error HTTP 400 Bad Request.

Automatización: En ASP.NET Core, los formularios generados mediante Tag Helpers (`<form>`) inyectan este token de forma automática sin que el programador tenga que añadir etiquetas adicionales, brindando protección por defecto [22].

ASP.NET Core Identity

Para la gestión de accesos, Microsoft provee ASP.NET Core Identity, un sistema completo de membresía y gestión de usuarios diseñado bajo estándares empresariales modernos de seguridad [27], [29]. Identity administra un conjunto completo de funcionalidades listas para usar que cubren todo el ciclo de vida del usuario dentro del sistema:

Servicio de Identity	Propósito y Funcionamiento Técnico
Gestión de Cuentas	Controla el registro de usuarios, inicios de sesión, restablecimiento de contraseñas olvidadas y bloqueos automáticos de cuenta tras múltiples intentos fallidos (prevención de fuerza bruta) [29].
Almacenamiento Seguro	Encripta automáticamente las contraseñas utilizando algoritmos robustos de derivación de claves (<i>PBKDF2</i>) con valores de sal aleatorios únicos antes de guardarlas en la base de datos [27].
Autorización por Roles y Reclamaciones	Permite segmentar los accesos de la aplicación asignando a los usuarios Roles clásicos (ej. <i>Admin</i> , <i>Cliente</i>) o Claims (reclamaciones específicas de identidad como <i>FechaDeNacimiento</i> o <i>PermisoEscritura</i>) [29].
Seguridad Avanzada	Soporta de forma nativa la autenticación de dos factores (2FA) a través de aplicaciones autenticadoras (como Google o Microsoft Authenticator) o servicios de SMS [27].

Tabla 14. Servicios de Identity.

Cabeceras de Seguridad HTTP

Además de la protección del código de la aplicación, el middleware de ASP.NET Core permite inyectar y configurar Cabeceras de Seguridad HTTP directo en la respuesta de la red. Estas cabeceras instruyen al navegador del cliente para que active barreras de defensa adicionales a nivel del sistema operativo o del motor de renderizado [27], [30]:

- **HSTS (HTTP Strict Transport Security):** Fuerza al navegador del usuario a comunicarse con el servidor utilizando única y exclusivamente conexiones cifradas HTTPS, denegando cualquier intento de conexión insegura HTTP.
- **X-Content-Type-Options (nosniff):** Evita que el navegador intente adivinar o inspeccionar el tipo MIME de los archivos descargados del servidor, mitigando ataques de suplantación de identidad de archivos ejecutables maliciosos disfrazados de imágenes.
- **Content-Security-Policy (CSP):** Controla estrictamente las fuentes de origen permitidas (scripts, estilos, imágenes) desde las cuales el navegador puede cargar recursos, actuando como la defensa definitiva contra ataques de Inyección de Scripts (Cross-Site Scripting o XSS).

5. TEMA 4: SEGURIDAD Y BUENAS PRÁCTICAS EN PROGRAMACIÓN WEB

5.1 Seguridad en Aplicaciones Web

La seguridad en el desarrollo web no debe considerarse una capa periférica que se añade al finalizar el proyecto; debe ser un pilar transversal integrado desde la fase de diseño mediante el principio de Defensa en Capas y el Principio de Mínimo Privilegio [31]. Este último dicta que cada componente de software, proceso o usuario de la aplicación debe poseer únicamente los permisos estrictamente necesarios para cumplir su función, limitando el radio de impacto en caso de una intrusión [32].

Autenticación vs. Autorización

Un error común en el diseño de software es confundir los mecanismos de control de acceso. Los estándares de seguridad los dividen en dos fases secuenciales e independientes [31]:

Concepto Técnico	¿Qué pregunta responde?	Mecanismo de Acción en el Servidor	Ejemplo Práctico
Autenticación (<i>Authentication / AuthN</i>)	¿Quién eres?	Proceso de verificación para confirmar que una identidad digital pertenece realmente al usuario que la reclama [33].	Validar que el usuario y la contraseña ingresados coincidan con el registro de la base de datos para otorgar un <i>token</i> de sesión.
Autorización (<i>Authorization / AuthZ</i>)	¿Qué tienes permitido hacer?	Ocurre después de la autenticación. El servidor evalúa los privilegios asignados a la identidad verificada para conceder o denegar el acceso a un recurso [34].	Verificar si el usuario autenticado tiene el rol de ADMIN antes de permitirle eliminar un registro del sistema.

Tabla 15. Autenticación vs Autorización (Fases secuenciales).

Falla Común: El diseño deficiente de la autorización da origen a una de las vulnerabilidades más críticas según el OWASP Top 10: el Control de Acceso Quebrado (Broken Access Control), que permite a usuarios comunes acceder a funciones administrativas [35].

Gestión Segura de Contraseñas: `password_hash()` / `Bcrypt`

Las contraseñas de los usuarios nunca deben almacenarse en texto plano ni utilizar algoritmos de encriptación bidireccional, ya que, si la base de datos se ve comprometida, las credenciales quedarían expuestas [31]. Tampoco se deben usar funciones hash rápidas como MD5 o SHA-1, las cuales son vulnerables a ataques de fuerza bruta modernos y tablas de búsqueda (Rainbow Tables) [33]. El estándar de la industria exige el uso de Funciones de Derivación de Claves lentas y adaptativas basadas en algoritmos como Bcrypt, Argon2 o PBKDF2 [33], [34].

El flujo criptográfico seguro implementa dos componentes clave:

- **Sal (Salt):** Una cadena de texto aleatoria que se genera automáticamente y se añade a la contraseña antes de aplicar el hash. Esto garantiza que, si dos usuarios eligen la misma contraseña, sus hashes resultantes en la base de datos serán completamente diferentes.
- **Factor de Costo:** Un parámetro que determina cuántas iteraciones matemáticas realizará el servidor para calcular el hash, ralentizando intencionalmente el proceso para volver inviables los ataques de fuerza bruta computacional.

Ejemplo:

```
// EJEMPLO EN PHP MODERNO: Uso nativo de Bcrypt
$passwordUsuario = $_POST['txt_password'];

// Genera un hash seguro y único aplicando automáticamente la sal y el costo
óptimo
$hashSeguro = password_hash($passwordUsuario, PASSWORD_BCRYPT, ['cost' => 12]);

// Verificación durante el inicio de sesión
if (password_verify($passwordUsuario, $hashSeguro)) {
    // Autenticación exitosa
}
```

Cifrado en Tránsito (TLS/SSL) y en Reposo

La protección de la información se divide según el estado en el que se encuentran los datos dentro del ecosistema de la aplicación:

- **Cifrado en Tránsito:** Protege los datos mientras viajan a través de la red pública (Internet) entre el navegador del cliente y el servidor web, mitigando ataques de interceptación (Man-in-the-Middle). Se implementa mediante el protocolo TLS (Transport Layer Security), el cual da vida a las conexiones HTTPS. Todo el tráfico, incluyendo cabeceras HTTP, cookies de sesión y datos de formularios, viaja cifrado criptográficamente en la capa de red [33].
- **Cifrado en Reposo:** Asegura la información cuando está almacenada estáticamente en sistemas persistentes (discos duros, bases de datos o almacenamiento en la nube). Protege el sistema en caso de robo físico del servidor o copias de seguridad. Se implementa cifrando directamente las columnas sensibles de la base de datos (como tarjetas de crédito) o utilizando cifrado a nivel de sistema de archivos (TDE - Transparent Data Encryption) [32].

Gestión Segura de Sesiones y Cookies

Dado que el protocolo HTTP no tiene estado, el servidor web depende de un identificador de sesión (Session ID) guardado en una cookie en el navegador para rastrear al usuario autenticado [33]. Si un atacante intercepta o roba esta cookie, puede suplantar completamente la identidad del usuario sin necesidad de conocer su contraseña (ataque de Secuestro de Sesión) [31], [35].

Para mitigar este riesgo, las cookies que transportan tokens o identificadores de sesión deben configurarse obligatoriamente con los siguientes atributos directos en el backend:

Atributo de Cookie	Función Técnica de Seguridad
HttpOnly	Bloquea por completo el acceso a la cookie a través de scripts de JavaScript del lado del cliente. Esto neutraliza el robo de sesiones en caso de que la aplicación sufra una vulnerabilidad de Inyección de Scripts (XSS) [33].
Secure	Instruye al navegador para que envíe la cookie única y exclusivamente a través de conexiones cifradas HTTPS . Impide que la cookie viaje en texto plano por redes Wi-Fi públicas desprotegidas [33].
SameSite	Controla si la cookie se envía junto con peticiones iniciadas desde sitios web externos. Configurado en Strict o Lax, actúa como la defensa principal del lado del cliente contra ataques de Falsificación de Solicitudes entre Sitios (CSRF) [33].

Tabla 16. Atributos de cookies y sus funciones.

5.2 OWASP Top 10 y Contramedidas

El OWASP Top 10 representa el estándar global sobre los riesgos de seguridad más críticos en aplicaciones web. Diseñar un backend seguro implica conocer cómo mitigar estas amenazas utilizando los mecanismos nativos de las tecnologías que hemos estudiado (PHP, Java y .NET) [34], [35].

A continuación, se presenta la matriz de vulnerabilidades y sus respectivas contramedidas implementadas a nivel de código y arquitectura:

#	Vulnerabilidad OWASP	Descripción Técnica	Contramedida en el Servidor (PHP / Java / .NET)
A01	Control de Acceso Quebrado (Broken Access Control)	Los usuarios pueden acceder a recursos o funciones fuera de sus permisos (escalabilidad de privilegios de forma horizontal o vertical).	Aplicar el Principio de Mínimo Privilegio. Usar filtros de autorización obligatorios (como @Authorize en Spring o hasRole() en .NET Identity). Denegar el acceso por defecto (<i>Deny by Default</i>).
A02	Fallas Criptográficas (Cryptographic Failures)	Exposición de datos sensibles en tránsito o en reposo debido a la falta de cifrado o al uso de algoritmos obsoletos (MD5, SHA1) [31].	Implementar TLS/HTTPS de forma obligatoria con cabeceras HSTS. Almacenar contraseñas usando algoritmos adaptativos lentos como Bcrypt (password_hash()). Cifrar datos en reposo usando AES-256.
A03	Inyección (Injection)	Datos suministrados por el usuario son interpretados por el intérprete como código ejecutable (Inyección SQL, Command Injection, LDAP) [35].	Uso estricto de Sentencias Preparadas (<i>Prepared Statements</i>) mediante PDO en PHP, Spring Data JPA en Java, o Entity Framework Core en .NET. Validar y sanitizar todas las entradas con listas blancas.

#	Vulnerabilidad OWASP	Descripción Técnica	Contramedida en el Servidor (PHP / Java / .NET)
A04	Diseño Inseguro (<i>Insecure Design</i>)	Defectos inherentes en la arquitectura y diseño del software antes de escribir el código fuente (falta de modelado de amenazas) [35].	Adoptar metodologías de desarrollo seguro como el ciclo de vida SSDC de NIST. Utilizar patrones de diseño y arquitecturas de referencia ya validadas y seguras.
A05	Configuración de Seguridad Incorrecta	Hardcoding de credenciales, visualización de errores detallados en producción o habilitación de servicios innecesarios [31].	Desactivar páginas de depuración en producción (ASPNETCORE_ENVIRONMENT=Production o display_errors = Off en PHP). Centralizar credenciales seguras fuera del código usando variables de entorno o <i>Secrets</i> .
A06	Componentes Vulnerables y Desactualizados	Construir software utilizando librerías, dependencias o frameworks antiguos que contienen fallas conocidas públicamente [33].	Utilizar gestores de paquetes oficiales (Composer, Maven, Gradle, NuGet) para actualizar dependencias constantemente. Escanear el árbol de dependencias con herramientas de composición de software (SCA).
A07	Fallas de Identificación y Autenticación	Debilidades en el manejo de sesiones, contraseñas débiles o vulnerabilidades ante ataques automatizados de fuerza bruta [31].	Forzar políticas de complejidad de contraseñas. Configurar cookies de sesión con atributos de aislamiento estricto: HttpOnly, Secure y SameSite. Implementar MFA y bloqueos automáticos de cuentas.
A08	Fallas en la Integridad de Datos y Software	Modificación no detectada de código, dependencias de repositorios no confiables o deserialización insegura de objetos (JSON/XML) [35].	Utilizar firmas digitales y sumas de verificación (<i>checksums</i>) para validar la integridad de las librerías. Evitar la deserialización directa de objetos provenientes de fuentes externas no confiables.
A09	Fallas en el Registro y Monitoreo de Seguridad	No registrar eventos críticos (como inicios de sesión fallidos) o auditorías insuficientes, impidiendo detectar o responder a hackeos activos [35].	Implementar un middleware de logging centralizado. Registrar intentos fallidos de autenticación y accesos denegados, asegurando que los logs se almacenen en servidores aislados y protegidos.
A10	Falsificación de Solicitudes del	Ocurre cuando la aplicación web backend	Validar y restringir las URLs de entrada mediante listas blancas de dominios permitidos.

#	Vulnerabilidad OWASP	Descripción Técnica	Contramedida en el Servidor (PHP / Java / .NET)
	Lado del Servidor (SSRF)	recibe una URL externa proporcionada por el usuario y la lee o descarga sin validar su destino [35].	Aislar la red del servidor backend para impedir que realice peticiones hacia recursos de la red interna corporativa (localhost, intranets).

Tabla 17. Vulnerabilidades OWASP y contramedidas implementadas.

5.3 Buenas Prácticas de Codificación Web

El desarrollo seguro de software moderno exige sustituir el modelo reactivo (corregir parches tras un hackeo) por un modelo proactivo. Esto se logra integrando la seguridad desde el inicio del ciclo de vida del desarrollo a través de prácticas de codificación defensiva y la cultura DevSecOps, la cual automatiza auditorías de seguridad en cada fase del despliegue [36].

Validación y Sanitización de Entradas

Cualquier dato proveniente de un cliente externo (formularios, parámetros de URL, cabeceras o JSON) debe considerarse no confiable por defecto [34]. Para neutralizar código malicioso antes de que toque las capas internas de la aplicación, se aplican dos filtros secuenciales [32]:

- **Validación (Filtro de Integridad):** Comprueba que el dato recibido cumpla estrictamente con un formato, tipo, longitud y rango específicos mediante listas blancas (allow-lists) [33]. Si el dato no cumple, se rechaza de inmediato (ej. verificar que un campo de edad contenga solo números entre 1 y 120).
- **Sanitización (Limpieza de Datos):** Modifica el dato de entrada para eliminar o neutralizar caracteres potencialmente peligrosos que no se pueden rechazar (ej. eliminar etiquetas `<script>` de un campo de comentarios de texto libre) [34].

Prevención de XSS: Escape de Salida y Content Security Policy (CSP)

El ataque de Secuestro de Scripts entre Sitios (XSS) ocurre cuando un atacante logra inyectar código JavaScript malicioso en las páginas web que ven otros usuarios [35]. Para mitigar este riesgo en la capa de presentación, se implementan dos defensas complementarias [32], [34]:

1. Escape de Salida (Output Encoding)

Consiste en convertir los caracteres especiales del código en sus entidades equivalentes de texto plano justo antes de renderizarlos en el navegador [34]. Esto asegura que el motor del navegador interprete el texto estrictamente como contenido visual y nunca como código ejecutable [33].

Texto original con script: `<script>robarCookies()</script>`

Texto transformado (escapado): `<script>robarCookies()</script>`

2. Content Security Policy (CSP)

Es una cabecera de seguridad HTTP que actúa como un escudo perimetral del lado del cliente [33]. Restringe estrictamente desde qué dominios u orígenes autorizados puede el navegador descargar y ejecutar archivos de JavaScript, hojas de estilo o fuentes, bloqueando en seco cualquier script inyectado de forma maliciosa [35].

Token CSRF en Formularios

Como se analizó en la arquitectura de ASP.NET, la inclusión de un Token Anti-Forgery (CSRF) es una práctica obligatoria para todo formulario web que realice operaciones de escritura en el servidor (POST, PUT, DELETE) [34].

El framework web debe generar un identificador criptográfico único por sesión y por petición. El cliente debe devolver este token en las cabeceras HTTP o en el cuerpo del formulario para que el servidor verifique la legitimidad del origen, impidiendo que sitios maliciosos exploten la sesión activa del usuario [32], [33].

Cabeceras de Seguridad HTTP

La configuración del servidor web (Apache, Nginx, IIS) o del middleware de la aplicación debe inyectar cabeceras HTTP específicas en cada respuesta de red para activar las defensas nativas de los navegadores web modernos [34]:

Cabecera HTTP	Función Técnica de Protección	Riesgo que Mitiga
Strict-Transport-Security (<i>HSTS</i>)	Fuerza al navegador a comunicarse con el servidor exclusivamente a través de canales cifrados HTTPS [33].	Ataques de degradación de protocolo y secuestro de tráfico (<i>SSL Stripping</i>) [35].
X-Frame-Options	Controla si la página web puede ser embebida dentro de etiquetas <frame>, <iframe> o <embed> de otros sitios web [34].	Ataques de Clickjacking (donde se engaña al usuario para que haga clic en un botón invisible) [31].
X-Content-Type-Options	Configurada con el valor nosniff, obliga al navegador a respetar estrictamente el tipo de contenido definido por el backend [33].	Ejecución maliciosa de archivos ocultos (<i>MIME sniffing</i> / <i>Drive-by downloads</i>) [33].

Tabla 18. Cabeceras HTTP (Activación de defensas nativas en navegadores modernos).

Auditoría y Logging de Eventos de Seguridad

El monitoreo constante es la única forma de detectar incidentes antes de que se conviertan en desastres. La aplicación debe contar con una infraestructura de registros (Logging) que documente eventos críticos del sistema en tiempo real [32]:

- **Eventos a registrar:** Autenticaciones fallidas, cambios de contraseñas, bloqueos de cuentas, intentos de acceso denegados por falta de privilegios y fallas críticas del sistema [34].
- **Integridad de los registros:** Los mensajes de log nunca deben almacenar datos sensibles del usuario (como contraseñas en texto plano, tokens de sesión o tarjetas de crédito) [32].
- **Seguridad de almacenamiento:** Siguiendo la normativa NIST SP 800-53, los archivos de registro deben enviarse de forma asíncrona hacia un servidor de almacenamiento centralizado y aislado para evitar que un atacante borre las huellas de su intrusión si el servidor web principal se ve comprometido [32].

6. APLICACIÓN DINÁMICA: CRUD CON AUTENTICACIÓN

6.1 Descripción de la Aplicación

La aplicación desarrollada consiste en un Sistema de Gestión de Inventario Seguro (CRUD) integrado con un módulo de control de accesos e identidades. El objetivo principal del proyecto es materializar de forma práctica los controles de mitigación contra las vulnerabilidades más críticas descritas en el estándar OWASP Top 10 (como inyecciones SQL, Cross-Site Scripting y Cross-Site Request Forgery). El sistema permite a los usuarios autenticados realizar operaciones de creación, lectura, actualización y eliminación de registros (productos) en un entorno controlado, aislando los datos de cada sesión y garantizando la integridad de la persistencia de datos.

Tecnologías Utilizadas

Para demostrar la versatilidad de los controles defensivos en diferentes entornos de ingeniería de software, la aplicación fue implementada bajo dos ecosistemas tecnológicos independientes:

Entorno PHP (Lado del Servidor Local):

- Lenguaje de Programación: PHP 8.x (Enfoque orientado a objetos).
- Extensión de Base de Datos: MySQLi con Sentencias Preparadas nativas.
- Servidor Web y Persistencia: Servidor HTTP Apache y motor de bases de datos MariaDB/MySQL, administrados a través de la suite local XAMPP.
- Frontend: HTML5 semántico, CSS3 estructurado para el diseño de componentes (tarjetas, formularios y contenedores) y renderizado del lado del servidor (SSR).

Entorno ASP.NET Core (Ecosistema Empresarial):

- Framework y Lenguaje: ASP.NET Core utilizando C# bajo el patrón arquitectónico MVC (Modelo-Vista-Controlador).
- Mapeador Objeto-Relacional (ORM): Entity Framework Core (EF Core) para el aislamiento y parametrización automática de la capa de datos.
- Gestión de Seguridad: ASP.NET Core Identity para el manejo cifrado de credenciales y middlewares nativos para la inyección de directivas antifalsificación (Antiforgery Tokens).

Funcionalidades Implementadas

El sistema cuenta con un alcance funcional cerrado y robusto, compuesto por los siguientes módulos operativos:

- Módulo de Registro de Usuarios (register.php): Permite el alta de nuevas identidades evaluando políticas de complejidad en las contraseñas (longitud mínima) y aplicando un algoritmo de dispersión irreversible (Bcrypt) antes del almacenamiento.
- Módulo de Autenticación Centralizado (login.php): Valida las credenciales de los usuarios utilizando verificación criptográfica, destruye los identificadores de sesión antiguos para evitar el secuestro de sesiones e inicializa el Token CSRF maestro en la memoria del servidor.

- Filtro de Control de Acceso (auth.php): Middleware perimetral que intercepta las rutas privadas del sistema (como el formulario de creación y el tablero general). Si un usuario intenta acceder de forma anónima, el sistema aborta la petición y lo redirige al login.
- Tablero Principal (dashboard.php): Interfaz de lectura que recupera los registros vinculados al usuario de forma segura y renderiza las tablas de datos utilizando entidades HTML codificadas.
- Operación de Creación (create.php): Formulario transaccional que valida la procedencia legítima de la solicitud mediante tokens sincronizados e inserta nuevos registros de productos (nombre y precio) a través de canales parametrizados estrictos.

6.2 Implementación en PHP

La solución práctica desarrollada en PHP adopta una arquitectura monolítica con renderizado del lado del servidor (SSR). Siguiendo las directrices del marco de trabajo de desarrollo seguro, el código rechaza el uso de funciones heredadas u obsoletas, implementando en su lugar la extensión MySQLi con un enfoque estrictamente preparado y funciones criptográficas de última generación para la gestión de identidades y la persistencia de datos.

Gestión de Identidades y Autenticación Segura (login.php y register.php)

La solución práctica desarrollada en el entorno PHP adopta una arquitectura monolítica basada en el Renderizado del Lado del Servidor (SSR). La totalidad del flujo de datos ha sido estructurada utilizando la extensión MySQLi mediante un enfoque estrictamente orientado a objetos. Siguiendo las directrices internacionales de desarrollo seguro, se ha rechazado el uso de funciones heredadas o la concatenación directa de cadenas de texto en las consultas, garantizando la integridad de la capa de persistencia y la gestión de identidades en el servidor local Apache bajo la suite XAMPP.

Por otro lado, el archivo login.php se encarga de la verificación segura mediante password_verify(). Al comprobarse la autenticidad de la credencial, el backend ejecuta de forma inmediata session_regenerate_id(true), un mecanismo defensivo crítico que invalida el identificador de sesión antiguo y emite uno nuevo, neutralizando por completo los riesgos de Fijación de Sesión (Session Fixation). Es en este punto exacto donde también nace el Token CSRF maestro que protegerá las transacciones posteriores del ciclo de vida de la sesión.

```
<?php
// login.php - Procesamiento Seguro de Autenticación y Control de Sesiones
session_start();
require 'db.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header("Location: login.html");
    exit();
}

$username = trim($_POST['username'] ?? '');
$password = $_POST['password'] ?? '';

if (empty($username) || empty($password)) {
    header("Location: login.html?error=empty");
    exit();
}
```

```
// 1. Mitigación de Inyección SQL mediante Sentencias Preparadas (Prepared
Statements)
$stmt = $conn->prepare("SELECT id, username, password FROM users WHERE username
= ?");
$stmt->bind_param("s", $username);
$stmt->execute();
$result = $stmt->get_result();

if ($result->num_rows === 1) {
    $user = $result->fetch_assoc();

    // 2. Verificación criptográfica del Hash Bcrypt
    if (password_verify($password, $user['password'])) {

        // 3. Control OWASP: Mitigación de Fijación de Sesión
        session_regenerate_id(true);

        $_SESSION['user_id'] = $user['id'];
        $_SESSION['username'] = $user['username'];

        // 4. Control OWASP: Generación del Token CSRF Maestro para la sesión
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));

        header("Location: dashboard.php");
        exit();
    }
}

$stmt->close();
header("Location: login.html?error=invalid");
exit();
?>
```

Persistencia de Datos y Operaciones del CRUD Seguro (create.php)

El módulo encargado de la inserción de nuevos elementos al inventario (create.php) unifica los controles de validación en el servidor con los mecanismos de defensa perimetral de la sesión. Antes de interactuar con el motor de la base de datos, el backend intercepta la petición POST y somete el parámetro csrf_token enviado por el formulario a una validación de igualdad estricta contra el token almacenado en la sesión del servidor. Si los valores no coinciden o el campo está ausente, la solicitud es destruida inmediatamente, mitigando los ataques de Falsificación de Solicitudes en Sitios Cruzados (CSRF).

Una vez superado el filtro de seguridad, los datos del producto se procesan utilizando tipos de datos validados (is_numeric() y floatval()) y se insertan a través de una sentencia preparada con parámetros enlazados de manera estricta ("sd" para string y double respectivamente), aislando los datos del cliente de la lógica del compilador SQL.

```
<?php
// create.php - Inserción Segura en el CRUD con Validación CSRF y SQLi
require 'auth.php'; // Filtro de control de acceso perimetral
require 'db.php';
```



```

$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // 1. CONTROL OWASP: Validación estricta del Token CSRF antes de procesar
    $token_recibido = $_POST['csrf_token'] ?? '';
    if (empty($token_recibido) || $token_recibido !== ($_SESSION['csrf_token']
    ?? '')) {
        die("Error de seguridad: Solicitud CSRF detectada e invalidada.");
    }

    $nombre = trim($_POST['nombre'] ?? '');
    $precio = $_POST['precio'] ?? '';

    // Validaciones de integridad de tipos en el Servidor
    if (empty($nombre) || $precio === '') {
        $error = "Todos los campos son obligatorios.";
    } elseif (!is_numeric($precio) || $precio < 0) {
        $error = "El precio debe ser un número válido mayor o igual a 0.";
    } else {
        $precio = floatval($precio);

        // 2. CONTROL OWASP: Sentencia preparada para mitigar Inyecciones SQL
        (A03:2021)
        $stmt = $conn->prepare("INSERT INTO productos (nombre, precio) VALUES
        (?, ?)");
        $stmt->bind_param("sd", $nombre, $precio);

        if ($stmt->execute()) {
            $stmt->close();
            header("Location: dashboard.php?msg=created");
            exit();
        } else {
            $error = "Error al guardar el producto.";
        }
        $stmt->close();
    }
}
?>

```

Renderizado de la Interfaz de Usuario y Mitigación de XSS

El componente visual de presentación en el frontend de creación implementa un diseño semántico que hereda la sesión segura. Para cumplir de forma exhaustiva con el control de Secuestro de Datos / Inyección de Scripts (XSS), todos los valores que se imprimen dinámicamente de regreso en las etiquetas HTML (como los mensajes de error o la persistencia de los campos de texto tras un fallo de validación) son codificados rigurosamente utilizando la función `htmlspecialchars()`. Esto garantiza que cualquier carácter especial como `<` o `>` sea transformado en entidades HTML inofensivas, impidiendo la ejecución arbitraria de JavaScript en el navegador del cliente.

```
<body>
```

```

<header><h1><img alt="Icon of a document with a plus sign" data-bbox="235 95 255 110"/> Gestión de Productos</h1></header>
<div class="container">
  <div class="card">
    <h2>+ Nuevo Producto</h2>
    <?php if ($error): ?>
      <div class="error"><?php echo htmlspecialchars($error, ENT_QUOTES,
'UTF-8'); ?></div>
    <?php endif; ?>

    <form method="POST">
      <input type="hidden" name="csrf_token" value="<?php echo
htmlspecialchars($_SESSION['csrf_token'] ?? '', ENT_QUOTES, 'UTF-8'); ?>">

      <label>Nombre del Producto</label>
      <input type="text" name="nombre"
value="<?php echo htmlspecialchars($_POST['nombre'] ?? '',
ENT_QUOTES, 'UTF-8'); ?>"
placeholder="Ej: Camiseta azul" required autofocus>

      <label>Precio ($)</label>
      <input type="number" name="precio" step="0.01" min="0"
value="<?php echo htmlspecialchars($_POST['precio'] ?? '',
ENT_QUOTES, 'UTF-8'); ?>"
placeholder="Ej: 29.99" required>

      <div class="btns">
        <button type="submit">Guardar</button>
        <a href="dashboard.php" class="back">Cancelar</a>
      </div>
    </form>
  </div>
</div>
</body>

```

6.3 Implementación en ASP.NET Core

Para contrastar el paradigma de scripts independientes de PHP, se replicó la funcionalidad del CRUD utilizando el framework ASP.NET Core bajo el patrón arquitectónico MVC. En este entorno, la seguridad perimetral se delega al middleware nativo de la plataforma.

La persistencia se gestiona a través del ORM Entity Framework Core, el cual compila las transacciones abstrayendo el lenguaje SQL tradicional. Esto garantiza que cada consulta enviada al motor de MariaDB/MySQL sea parametrizada de manera nativa y matemática en el núcleo de ejecución, volviendo al sistema inmune frente a vectores de Inyección SQL sin necesidad de escribir enlazados manuales de parámetros.

A nivel de código, el controlador implementa la inyección de dependencias para el contexto de datos y segmenta las peticiones mediante verbos HTTP estrictos:

```

// Controllers/ProductosController.cs - Procesamiento del CRUD en .NET Core
using Microsoft.AspNetCore.Mvc;

```

```

using CrudDotNet.Data;
using CrudDotNet.Models;

namespace CrudDotNet.Controllers
{
    public class ProductosController : Controller
    {
        private readonly ApplicationDbContext _context;

        // Inyección de Dependencias del Contexto de Base de Datos
        public ProductosController(ApplicationDbContext context)
        {
            _context = context;
        }

        // Renderiza el formulario visual (Petición segura GET)
        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        // Procesa la inserción del producto (Petición transaccional POST)
        [HttpPost]
        [ValidateAntiForgeryToken] // CONTROL OWASP: Validación automatizada de
Token CSRF
        public async Task<IActionResult> Create(Producto producto)
        {
            // Evaluación del estado del modelo según reglas de integridad
predefinidas
            if (ModelState.IsValid)
            {
                // CONTROL OWASP: Inserción parametrizada automatizada por
Entity Framework
                _context.Add(producto);
                await _context.SaveChangesAsync();

                return RedirectToAction("Index", "Home");
            }

            // Si el modelo es inválido, retorna a la vista sanitizada con los
errores

            return View(producto);
        }
    }
}

```

Asimismo, la capa de presentación (Create.cshtml) aprovecha el motor de vistas Razor. Al invocar los Tag Helpers nativos (como asp-action), el framework intercepta el renderizado e inyecta dinámicamente un campo oculto con un token criptográfico dual (antifalsificación), sincronizado con una cookie cifrada en el

navegador. Cualquier intento de alterar esta estructura levanta una excepción inmediata en el servidor Kestrel.

@model CrudDotNet.Models.Producto

```
<div class="container mt-5">
  <div class="card shadow">
    <div class="card-header bg-primary text-white">
      <h3>+ Nuevo Producto (.NET Core)</h3>
    </div>
    <div class="card-body">
      <form asp-action="Create" method="post">
        <div class="form-group mb-3">
          <label asp-for="Nombre" class="form-label">Nombre del
Producto</label>
          <input asp-for="Nombre" class="form-control" required />
          <span asp-validation-for="Nombre" class="text-
danger"></span>
        </div>
        <div class="form-group mb-4">
          <label asp-for="Precio" class="form-label">Precio
($)</label>
          <input asp-for="Precio" type="number" step="0.01"
class="form-control" required />
          <span asp-validation-for="Precio" class="text-
danger"></span>
        </div>
        <button type="submit" class="btn btn-success">Guardar
Producto</button>
      </form>
    </div>
  </div>
</div>
```

6.4 Controles OWASP Implementados

Para garantizar que la aplicación dinámica desarrollada sea resiliente frente a los vectores de ataque más críticos de la industria, se integraron controles defensivos basados de forma estricta en el estándar OWASP Top 10. A continuación, se detalla la correspondencia analítica e implementación de cada contramedida en los entornos de programación analizados:

1. Prevención de Secuestro de Datos / Scripting en Sitios Cruzados (OWASP A03:2021 - Inyección / XSS)

El ataque Cross-Site Scripting (XSS) se neutraliza mediante la técnica de Codificación de Salida (Output Encoding). Este proceso intercepta los datos provenientes de la base de datos o de peticiones de usuario antes de ser renderizados en el navegador del cliente, convirtiendo los caracteres con significado sintáctico en HTML en entidades de texto plano inofensivas.

Tecnología	Mecanismo de Implementación	Propósito Técnico
PHP	<code>htmlspecialchars(\$data, ENT_QUOTES, 'UTF-8')</code>	Convierte caracteres como <code><</code> , <code>></code> , <code>&</code> , <code>"</code> y <code>'</code> en equivalentes seguros (ej. <code>&lt;</code>), impidiendo que el navegador ejecute etiquetas <code><script></code> .
ASP.NET Core	<code>@variable</code> (Razor Contextual Encoding) / <code>HtmlEncoder</code>	El motor de vistas Razor codifica por defecto toda cadena de texto de forma automática según el contexto (HTML, Atributo o JavaScript).

Tabla 19. Prevención de secuestro de datos en php y asp.net core.

2. Mitigación de Falsificación de Solicitudes en Sitios Cruzados (OWASP A01:2021 - Control de Acceso Quebrado)

El ataque CSRF obliga al navegador de una víctima autenticada a enviar una solicitud maliciosa no deseada a una aplicación web. El control implementado mitiga este riesgo mediante el patrón de Token Sincronizador, vinculando un identificador pseudoaleatorio criptográficamente fuerte a la sesión del usuario.

Tecnología	Mecanismo de Implementación	Propósito Técnico
PHP	<code>bin2hex(random_bytes(32))</code> y verificación condicional <code>\$_POST</code>	Produce una cadena única en el login guardada en <code>\$_SESSION</code> . Al recibir un POST (como en <code>create.php</code>), compara de forma estricta el token del formulario con el del servidor; si no coinciden, aborta la petición.
ASP.NET Core	<code>@Html.AntiForgeryToken()</code> / <code>[ValidateAntiForgeryToken]</code>	Genera y valida un token dual (una cookie cifrada y un campo oculto) de forma automatizada a través del middleware de seguridad de la plataforma .NET.

Tabla 20. Mitigación de Falsificación de Solicitudes en Sitios Cruzados en php y asp.net core.

3. Almacenamiento Criptográfico de Credenciales (OWASP A02:2021 - Fallas Criptográficas)

Para proteger la identidad de los usuarios frente a fugas de información o accesos no autorizados a la base de datos, las contraseñas jamás se almacenan en texto plano. Se aplican funciones de dispersión (hashing) criptográfica con factor de trabajo adaptativo y adición automática de sal (salt).

Tecnología	Mecanismo de Implementación	Propósito Técnico
PHP	<code>password_hash(\$pass, PASSWORD_BCRYPT)</code>	Utiliza el algoritmo Bcrypt para generar una cadena irreversible de 60 caracteres. La verificación en el inicio de sesión se delega a la función segura <code>password_verify()</code> .

Tecnología	Mecanismo de Implementación	Propósito Técnico
ASP.NET Core	PasswordHasher<TUser> (ASP.NET Core Identity)	Implementa por defecto el algoritmo PBKDF2 con HMAC-SHA256 y un conteo iterativo dinámico que ralentiza intencionalmente los ataques de fuerza bruta.

Tabla 21. Almacenamiento Criptográfico de Credenciales en php y asp.net core.

4. Parametrización de Consultas / Sentencias Preparadas (OWASP A03:2021 - Inyección SQL)

La Inyección SQL (SQLi) se erradica por completo separando radicalmente la lógica de la consulta de los datos proporcionados por el usuario. Al precompilar la estructura SQL en el motor de la base de datos, los parámetros se tratan estrictamente como valores literales, nunca como código ejecutable.

Tecnología	Mecanismo de Implementación	Propósito Técnico
PHP	\$conn->prepare() y \$stmt->bind_param()	Enlaza los parámetros indicando explícitamente su tipo de datos (ej. "sd" para texto y flotante). Esto garantiza que caracteres maliciosos como ' OR '1'='1 pierdan su semántica de control.
ASP.NET Core	LINQ / Entity Framework Core (.SaveChangesAsync())	Modela las consultas mediante un Mapeador Objeto-Relacional (ORM). EF Core parametriza de forma nativa y automática todas las peticiones SQL enviadas al motor.

Tabla 22. Parametrización de Consultas / Sentencias Preparadas en php y asp.net core.

6.5 Análisis Comparativo de Entornos Tecnológicos

Para fundamentar la selección de la infraestructura de software, se presenta una evaluación crítica y objetiva basada en ocho criterios arquitectónicos. Esta comparativa contrasta el entorno ágil monolítico de PHP frente a la plataforma empresarial robusta de .NET:

Criterio Técnico	Entorno PHP (Nativo / XAMPP)	Entorno ASP.NET Core (C#)	Impacto en la Aplicación
1. Seguridad por Defecto	Bajo / Manual. Requiere que el desarrollador implemente explícitamente funciones como htmlspecialchars() o validaciones CSRF manuales para evitar fallas.	Alto / Integrado. Los componentes nativos (Razor, middlewares) bloquean XSS, CSRF y secuestro de sesiones de forma automática si no se configuran en contra.	En PHP el riesgo de error humano es mayor; en .NET la plataforma actúa como un escudo perimetral automatizado.
2. Arquitectura Base	Monolítica basada en Scripts. Cada archivo .php suele procesar la	Patrón MVC Estricto. Separación formal de responsabilidades en	.NET ofrece un acoplamiento débil que facilita el

Criterio Técnico	Entorno PHP (Nativo / XAMPP)	Entorno ASP.NET Core (C#)	Impacto en la Aplicación
	lógica de control y la presentación visual en el mismo entorno de ejecución.	carpetas estructuradas de Modelos, Vistas y Controladores.	mantenimiento a gran escala; PHP destaca en despliegues rápidos.
3. Acceso a Datos	Manual / Procedimental. Depende de extensiones como mysqli o PDO, donde el desarrollador redacta las sentencias SQL de forma explícita.	Abstracción por ORM. Utiliza <i>Entity Framework Core</i> , interactuando con la base de datos mediante objetos C# y consultas LINQ parametrizadas.	EF Core elimina el riesgo de consultas mal estructuradas; MySQLi otorga control absoluto sobre el rendimiento de la consulta.
4. Rendimiento y Concurrencia	Moderado. Basado en un modelo de ejecución por hilos independientes por petición (<i>Thread-per-request</i>) gestionado habitualmente por Apache.	Alto / Excelente. Motor de ejecución asíncrono basado en el servidor <i>Kestrel</i> , optimizado para procesar decenas de miles de peticiones por segundo.	.NET exhibe una latencia menor bajo un alto volumen de transacciones concurrentes en el CRUD.
5. Manejo de Sesiones	Estado en Servidor / Archivos. Guarda los datos en el sistema de archivos local (/tmp) referenciados por una cookie PHPSESSID.	Middleware de Sesión Distribuido. Permite almacenar estados en memoria, bases de datos o clústeres de caché (Redis) de forma transparente.	.NET escala de forma nativa en entornos de nube distribuidos; PHP requiere configuraciones adicionales en su php.ini.
6. Curva de Aprendizaje	Baja / Inmediata. Sintaxis flexible, tipado dinámico y un despliegue directo sin necesidad de compilación previa.	Media / Alta. Requiere comprender tipado estricto (C#), programación asíncrona, inyección de dependencias y el ciclo de vida del framework.	PHP permite construir y corregir el CRUD en menor tiempo; .NET demanda mayor inversión de tiempo inicial en diseño arquitectónico.
7. Ecosistema y Comunidad	Masivo y Heredado. Domina más del 75% de las webs activas. Abundancia de documentación, foros de soporte y hosting económicos compatibles con XAMPP.	Corporativo y Moderno. Respaldado por Microsoft y una comunidad de código abierto madura, enfocado en software empresarial y entornos corporativos.	Es extremadamente sencillo encontrar soluciones a errores comunes en PHP; .NET ofrece paquetes de dependencias altamente estandarizados (NuGet).
8. Multiplataforma y Despliegue	Nativo. Altamente portable. Se ejecuta idénticamente en Windows, Linux o	Nativo (Moderno). A partir de .NET Core, el framework es completamente	Ambas tecnologías garantizan que la aplicación pueda migrar del entorno de

Criterio Técnico	Entorno PHP (Nativo / XAMPP)	Entorno ASP.NET Core (C#)	Impacto en la Aplicación
	macOS mediante intérpretes estándar del lado del servidor.	multiplataforma y está optimizado para su empaquetado en contenedores Docker.	desarrollo local a producción sin alterar el código fuente.

7. CONCLUSIONES

La culminación de este análisis comparativo y el desarrollo práctico de la aplicación dinámica permiten concluir que el éxito en la ingeniería de software web no depende de la adopción ciega de lenguajes de programación de última tendencia, sino de la comprensión profunda de los modelos de ejecución arquitectónicos y los principios de diseño seguro subyacentes. Tecnologías como PHP se mantienen a la vanguardia global debido a su constante evolución de rendimiento (gracias a motores como el JIT) y la simplificación de su despliegue económico. Por otro lado, ecosistemas fuertemente tipados como Java (a través de Spring Boot) y C# (mediante ASP.NET Core) se consolidan como los estándares indiscutibles para el ámbito empresarial y las infraestructuras de la nube (cloud-native), ofreciendo herramientas avanzadas de inyección de dependencias, automatización de compilación y robustas capas de abstracción para persistencia de datos.

Asimismo, la investigación evidencia que la seguridad de las aplicaciones web debe ser abordada bajo una filosofía de diseño proactivo (seguridad por diseño) y nunca como un parche reactivo en fases avanzadas de desarrollo. Al integrar los controles estipulados por el estándar OWASP Top 10 dentro de los pipelines de frameworks modernos como la validación automatizada mediante Data Annotations en .NET, la tokenización nativa contra vulnerabilidades CSRF o las capas de seguridad perimetral a través de Spring Security y cabeceras HTTP se mitiga drásticamente la superficie de exposición de los sistemas frente a actores maliciosos.

En última instancia, el desarrollo práctico del sistema CRUD con autenticación sirve como validación empírica de estos conceptos. Demuestra de manera fehaciente que, independientemente del lenguaje de programación elegido (sea este PHP, Java o .NET), las buenas prácticas de codificación tales como el aislamiento estricto de los datos del cliente mediante sentencias preparadas, la sanitización de entradas, el escape de salida y el control de accesos basado en roles son transversales y universales. La adopción sistemática de estos criterios metodológicos es lo que distingue a una aplicación web comercialmente viable de un software vulnerable, garantizando sistemas backend resilientes, altamente escalables y técnicamente alineados con los estándares que exige la industria del software.

9. REFERENCIAS

- [1] University of California, “Modelos de ejecución en el lado del servidor,” CSE 135 Course Notes. Accessed: Jun. 17, 2026. [Online]. Available: <https://cse135.site/models.html>
- [2] L. E. . Ullman, *PHP and MySQL for dynamic web sites*. Peachpit Press, 2018.
- [3] Michael Beck, “2025 Año en Reseña: Servidor Side Tecnologías,” Beck Digital. Accessed: Jun. 17, 2026. [Online]. Available: <https://beckdigital.com/insights/2025-year-in-review-server-side-technologies/>
- [4] J. Zhao, K. Zhu, L. Yu, H. Huang, and Y. Lu, “Yama: Precise Opcode-Based Data Flow Analysis for Detecting PHP Applications Vulnerabilities,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 7748–7763, 2025, doi: 10.1109/TIFS.2025.3592537.
- [5] Randal. Schwartz, brian. foy, and Tom. Phoenix, *Learning Perl, 8th Edition*. O’Reilly Media, Inc., 2021.
- [6] The Perl Foundation, “Perl Documentation,” The Perl Foundation. Accessed: Jun. 17, 2026. [Online]. Available: <https://www.perl.org/cpan.html>
- [7] Matthew Weier O’Phinney and Connor Penhale, “PHP vs. Perl: Comparación de rendimiento y características clave,” Zend. Accessed: Jun. 17, 2026. [Online]. Available: <https://www.zend.com/blog/php-vs-perl>
- [8] Bert. Bates and Kathy. Sierra, *Head First Java, 3rd Edition*. O’Reilly Media, Inc., 2021.
- [9] Oracle Corporation, “Visión general de la tecnología de máquinas virtuales en Java,” Java Platform. Accessed: Jun. 17, 2026. [Online]. Available: <https://docs.oracle.com/en/java/javase/21/vm/java-virtual-machine-technology-overview.html>
- [10] Oracle Corporation, “JDK 21 Documentation,” Java Platform. Accessed: Jun. 17, 2026. [Online]. Available: <https://docs.oracle.com/en/java/javase/21/index.html>
- [11] Eclipse Foundation, “Andén EE de Yakarta 11,” Jakarta EE Platform Specification. Accessed: Jun. 17, 2026. [Online]. Available: <https://jakarta.ee/specifications/platform/11/>
- [12] Apache Software Foundation, “Documentacion de Maven,” Apache Maven Project Documentation. Accessed: Jun. 17, 2026. [Online]. Available: <https://maven.apache.org/>
- [13] Gradle Inc, “Manual de usuario de Gradle,” Gradle User Manual. Accessed: Jun. 18, 2026. [Online]. Available: <https://docs.gradle.org/current/userguide/userguide.html>
- [14] Apache Software Foundation, “Documentacion de Apache Tomcat 11,” Tomcat Apache. Accessed: Jun. 18, 2026. [Online]. Available: <https://tomcat.apache.org/tomcat-11.0-doc/index.html>
- [15] Red Hat, “Documentación de WildFly,” WildFly. Accessed: Jun. 18, 2026. [Online]. Available: <https://docs.wildfly.org/36/>
- [16] Eclipse Foundation, “Eclipse Jetty 12,” Jetty. Accessed: Jun. 18, 2026. [Online]. Available: <https://jetty.org/docs/jetty/12/index.html>
- [17] Eclipse Foundation, “Yakarta Servlet 6.1,” Jakarta. Accessed: Jun. 18, 2026. [Online]. Available: <https://jakarta.ee/specifications/servlet/6.1/>
- [18] Eclipse Foundation, “Yakarta Páginas 4.0,” Jakarta. Accessed: Jun. 18, 2026. [Online]. Available: <https://jakarta.ee/specifications/pages/4.0/>
- [19] VMware, “Spring Framework Documentation,” Spring. Accessed: Jun. 18, 2026. [Online]. Available: <https://docs.spring.io/spring-framework/reference/index.html>
- [20] Craig. Walls, *Spring Boot in action*. Manning Publications, 2016.
- [21] Craig. Walls, *Spring in action*. Manning Publications Co., 2022.
- [22] Adam. Freeman, *Pro ASP.NET Core 6 : develop cloud-ready web applications using MVC, Blazor, and Razor Pages*. Apress, 2022.
- [23] Microsoft, “ASP.NET Core Overview,” Microsoft Learn. Accessed: Jun. 18, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-10.0&preserve-view=true>

- [24] Dino. Esposito, *Programming ASP.NET Core*. Microsoft : Pearson Education, 2018.
- [25] Adam. Freeman, *Pro ASP.NET Core 3 : develop cloud-ready web applications using MVC, Blazor, and Razor Pages*. Apress L.P., 2020.
- [26] Dino. Esposito, *Programming ASP.NET Core*. Microsoft : Pearson Education, 2018.
- [27] Microsoft, “ASP.NET documentation,” Microsoft Learn. Accessed: Jun. 18, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-9.0>
- [28] Microsoft, “Arquitectura y conceptos de Razor Pages en ASP.NET Core,” Microsoft Learn. Accessed: Jun. 18, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/razor-pages/?view=aspnetcore-10.0&preserve-view=true&tabs=visual-studio>
- [29] Microsoft, “ASP.NET Temas básicos de seguridad,” Microsoft Learn. Accessed: Jun. 18, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/?view=aspnetcore-9.0>
- [30] Microsoft, “Validación de modelos en ASP.NET Core MVC y Razor Pages,” Microsoft Learn. Accessed: Jun. 18, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation?view=aspnetcore-9.0>
- [31] Andrew. Hoffman, *Web application security : exploitation and countermeasures for modern web applications*. O’Reilly Media, Inc., 2024.
- [32] “Security and Privacy Controls for Information Systems and Organizations,” Gaithersburg, MD, Sep. 2020. doi: 10.6028/NIST.SP.800-53r5.
- [33] Mike. Harwood and Ron. Price, *Internet and web application security*. Jones & Bartlett Learning, 2024.
- [34] OWASP Foundation, “OWASP Developer Guide,” OWASP Foundation. Accessed: Jun. 18, 2026. [Online]. Available: <https://devguide.owasp.org/>
- [35] OWASP Foundation, “OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks, 2021,” OWASP Foundation. Accessed: Jun. 18, 2026. [Online]. Available: <https://owasp.org/Top10/2021/>
- [36] M. Souppaya, K. Scarfone, and D. Dodson, “Secure Software Development Framework (SSDF) version 1.1 ;,” Feb. 2022. doi: 10.6028/NIST.SP.800-218.

10. ANEXOS

Anexo A: Commits – GitHub

Commits

main

All users

All time

Commits on Jun 19, 2026

Merge branch 'main' of <https://github.com/carla22072004/Practica-EXPERIMENTAL-II>

1312642

DarwinSM21 committed 5 minutes ago

Merge remote-tracking branch 'origin/main'

carla22072004 committed 34 minutes ago

fecbcb1

reorganizar funciones de login

carla22072004 committed 34 minutes ago

ca80445

Merge remote-tracking branch 'origin/main'

carla22072004 committed 35 minutes ago

c3c79e2

Merge branch 'main' of <https://github.com/carla22072004/Practica-EXPERIMENTAL-II>

9b5884a

DarwinSM21 committed 35 minutes ago

cambios distintos en el crud

carla22072004 committed 36 minutes ago

ffb4ace

feat: Crear ASPNETCore e implementar modelo de bd

DarwinSM21 committed 41 minutes ago

7c49324

Merge remote-tracking branch 'origin/main'

carla22072004 committed 47 minutes ago

06258a4

formulario de cerracion

carla22072004 committed 47 minutes ago

95b864f

creacion del crud

carla22072004 committed 1 hour ago

f01d1dc

Merge remote-tracking branch 'origin/main'

carla22072004 committed 1 hour ago

1

4a8319a

feat: Implementar update, delete y read

DarwinSM21 authored and carla22072004 committed 1 hour ago

7e6f71

feat: Implementar update, delete y read

DarwinSM21 committed 1 hour ago

8bce700

Implementar dashboard, auth y logout

DarwinSM21 committed 2 hours ago

ee9ba64

Ordenar archivos en carpeta php_crud

DarwinSM21 committed 2 hours ago

9e4fb60

feat: Implementar registro, bd y create

DarwinSM21 committed 2 hours ago

9det27d

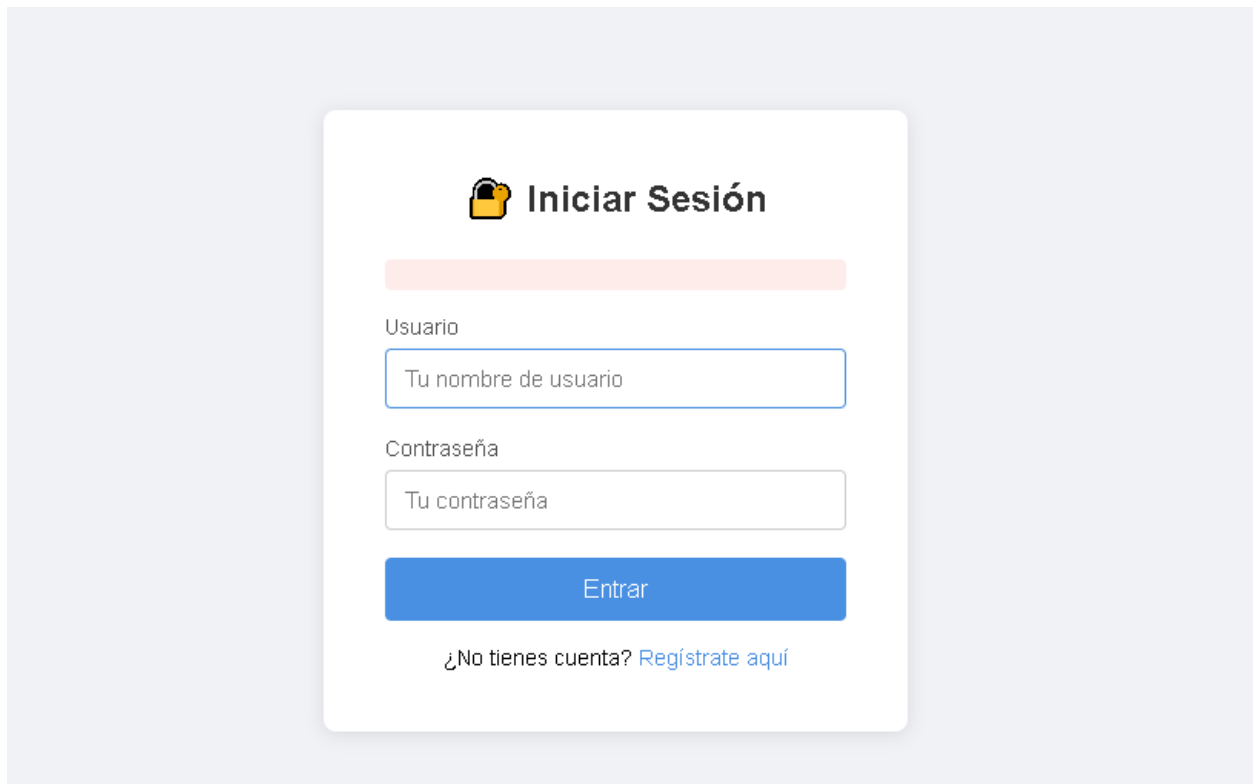
feat: Agregar login y estructurar base de datos

Verified

DarwinSM21 authored 2 hours ago

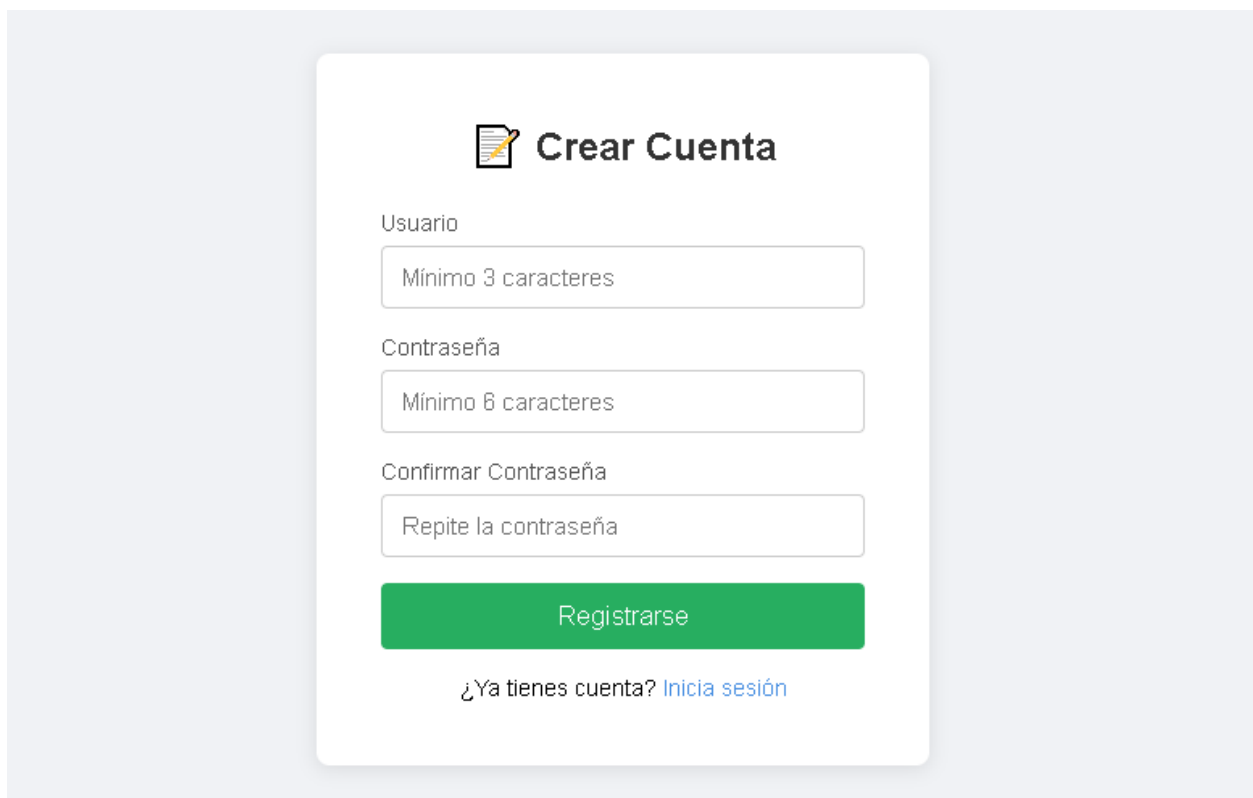
a5fc6a

Anexo B: Capturas de Pantalla de la Aplicación

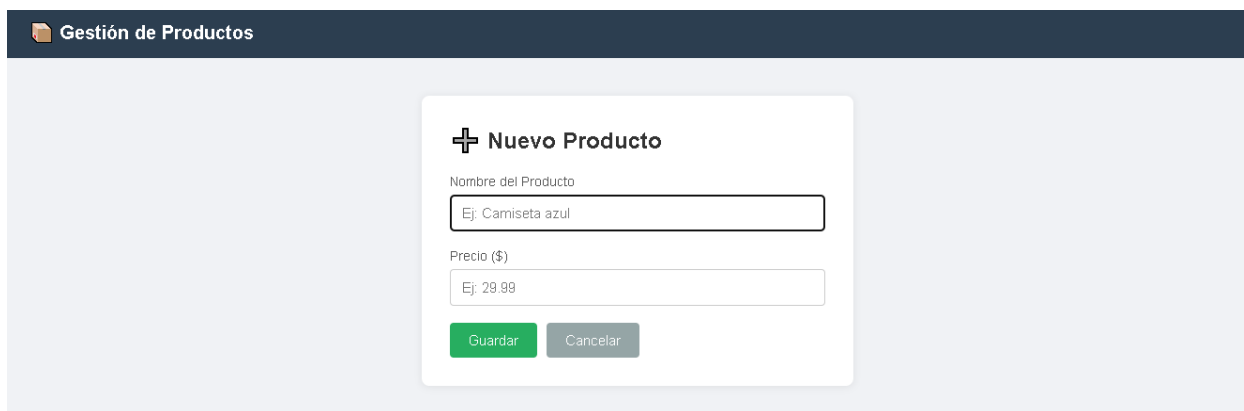


The screenshot shows a login form titled "Iniciar Sesión" with a padlock icon. It features a light orange header bar, a "Usuario" field with placeholder text "Tu nombre de usuario", a "Contraseña" field with placeholder text "Tu contraseña", and a blue "Entrar" button. At the bottom, there is a link: "¿No tienes cuenta? [Regístrate aquí](#)".

Fig. 3. Interfaz de login, PHP.



The screenshot shows a "Crear Cuenta" form with a notepad icon. It includes three input fields: "Usuario" with placeholder "Mínimo 3 caracteres", "Contraseña" with placeholder "Mínimo 6 caracteres", and "Confirmar Contraseña" with placeholder "Repite la contraseña". A green "Registrarse" button is at the bottom, followed by a link: "¿Ya tienes cuenta? [Inicia sesión](#)".

Fig. 4. Interfaz de registro, PHP.*Fig.5. Interfaz de gestión de productos, PHP.**Fig.6. Interfaz de registro de productos, PHP.**Fig.7. Interfaz de registro de productos, ASPNETCore.*